

**DEPARTMENT OF COMPUTING AND INFORMATION SCIENCE
COURSE OF STUDY: COMPUTER SCIENCE**

**DESIGN AND IMPLEMENTATION OF A WEB-BASED SPORTS EVENT
MANAGEMENT SYSTEM**

A PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF COMPUTING
AND INFORMATION SCIENCE IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE AWARD OF A DEGREE IN COMPUTER SCIENCE

BY

**OKUNBONADE ABIODUN AYOMIDE
MATRIC NUMBER: 5095**

OCTOBER, 2026

CERTIFICATION

This is to certify that this project report, titled “*Design and Implementation of a Web-Based Sports Event Management System*”, was carried out by **OKUNBONADE ABIODUN AYOMIDE** (Matric Number: **5095**) in the Department of Computing and Information Science. The report presents the analysis, design, implementation and evaluation of a web-based platform for the organisation, scheduling, score tracking and publication of sports events, and it has been read and approved as meeting the requirements for the award of a degree in Computer Science.

[SUPERVISOR'S NAME AND TITLE]

Project Supervisor

Date

[HEAD OF DEPARTMENT'S NAME AND TITLE] Date

Head of Department

External Examiner

Date

DECLARATION

I, **OKUNBONADE ABIODUN AYOMIDE**, hereby declare that this project report, titled “*Design and Implementation of a Web-Based Sports Event Management System*”, is a record of work carried out by me in the Department of Computing and Information Science. The system described in this report was designed, implemented and tested as part of the project, and all sources of information used have been duly acknowledged in the references. The test results reported in Chapter Four were obtained by running the implemented system; where an evaluation activity (such as the user questionnaire) had not yet been carried out, this is stated clearly rather than replaced with assumed figures.

OKUNBONADE ABIODUN AYOMIDE

Date

ACKNOWLEDGEMENT

I acknowledge the support of the Department of Computing and Information Science, lecturers, project supervisors, sports organisers, coaches, students and other stakeholders whose perspectives are relevant to the design of a practical sports event management platform. I also acknowledge the role of modern web technologies in making information systems more accessible for organisational activities.

ABSTRACT

Sports events contribute to physical fitness, teamwork, discipline, talent discovery and social interaction, yet their administration becomes difficult when registration, scheduling, score recording and result publication depend on paper records, spreadsheets and verbal communication. This project designed and implemented a Web-Based Sports Event Management System for educational institutions and small organisations, and deployed it as *BOUESTI Sports*, a sports portal for the internal student competitions of Bamidele Olumilua University of Education, Science and Technology, Ikere-Ekiti. The system is a centralised platform through which administrators manage seasons, competitions, teams, athletes, fixtures, scores, match events, line-ups, news and media, while participants and spectators view schedules, live and final scores, automatically computed standings, player statistics and published results. Students can also create accounts, apply for team trials and send enquiries. The development followed a system development research design with iterative and incremental implementation. The system uses a three-tier client-server architecture implemented with the Laravel 12 PHP framework (Model-View-Controller pattern), a MySQL/MariaDB relational database, and a responsive interface built with HTML, CSS, JavaScript, Blade templates, Tailwind CSS and Alpine.js, developed with Visual Studio Code and XAMPP. The system was verified with 96 automated feature and unit tests (921 assertions), all of which passed. The tests cover registration, authentication, role-based access, team and player management, fixture scheduling with conflict detection, score recording and result publication. Responsiveness was checked on all 27 public pages at ten screen widths from 320 to 1440 pixels, and common pages were served in a median of 43–82 milliseconds in the local test environment. The results show that a centralised web system removes manual computation of standings, prevents common data-entry errors, preserves historical records and makes sports information accessible on any device. A user-satisfaction questionnaire has been prepared for post-deployment evaluation.

Keywords: sports event management, web-based information system, fixture scheduling, score tracking, Laravel, MVC, relational database, three-tier architecture.

TABLE OF CONTENTS

CERTIFICATION	ii
DECLARATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF TABLES	ix
LIST OF FIGURES	x
LIST OF ABBREVIATIONS	xi
CHAPTER ONE: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Statement of the Problem	3
1.3 Motivation for the Study	3
1.4 Aim and Objectives	4
1.5 Research Questions	4
1.6 Significance of the Study	4
1.7 Scope of the Study	5
1.8 Limitations of the Study	6
1.9 Operational Definition of Terms	6
1.10 Organisation of the Report	7
CHAPTER TWO: LITERATURE REVIEW	8
2.1 Introduction	8
2.2 Concept of Sports Event Management	8
2.3 Sports Event Administration	8
2.4 Information Systems in Event Management	9
2.5 Web-Based Information Systems	9
2.6 Sports Registration and Team Management	10
2.7 Scheduling and Fixture Management	10
2.8 Score and Result Management	11
2.9 Database Management	11
2.10 Three-Tier Architecture and the MVC Pattern	12
2.11 Web Development Frameworks	12

2.12 Security, Privacy and Access Control	12
2.13 Usability in Sports Event Systems	13
2.14 Review of Related Approaches	13
2.15 Theoretical Perspective	14
2.16 Gap in Literature and Rationale for the Study	14
CHAPTER THREE: METHODOLOGY AND SYSTEM ANALYSIS	15
3.1 Research Design	15
3.2 Development Approach	15
3.3 Methods of Data Collection	16
3.4 Analysis of the Existing System	16
3.5 The Proposed System	16
3.6 Functional Requirements	17
3.7 Non-Functional Requirements	18
3.8 Feasibility Analysis	18
3.9 System Architecture	19
3.10 Use Case Model	20
3.11 Data Flow Design	21
3.12 Database Design	22
3.13 Algorithm Design	24
3.14 Input and Output Design	26
3.15 Development Tools and Technologies	26
3.16 Security and Data Integrity Design	26
3.17 Testing Strategy	27
CHAPTER FOUR: SYSTEM IMPLEMENTATION AND EVALUATION	28
4.1 Introduction	28
4.2 Implementation Environment	28
4.3 System Structure	28
4.4 Database Implementation and Test Data	29
4.5 User Interface Implementation	29
4.6 Authentication and Authorisation	34
4.7 Team and Athlete Management	36
4.8 Event and Fixture Management	39

4.9 Score and Result Management	40
4.10 Administrative Dashboard	41
4.11 Registration of Participants and Communication	42
4.12 System Integration	45
4.13 Test Cases and Results	45
4.14 Requirements Traceability	47
4.15 Usability Evaluation	47
4.16 Performance Evaluation	48
4.17 Discussion of Findings	48
CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS	50
5.1 Summary	50
5.2 Summary of Findings	50
5.3 Conclusion	51
5.4 Recommendations	51
5.5 Contribution to Knowledge	51
5.6 Data Quality and Record Integrity	52
5.7 Maintenance and Sustainability	52
5.8 Suggested Future Work	52
REFERENCES	54
APPENDIX A: DATA DICTIONARY	55
APPENDIX B: COMPLETED SYSTEM TEST SHEET	58
APPENDIX C: PROPOSED USER QUESTIONNAIRE	59
APPENDIX D: MODULE SPECIFICATION	60
APPENDIX E: ACCEPTANCE CRITERIA	61
APPENDIX F: USER AND INSTALLATION GUIDE	62
APPENDIX G: SELECTED SOURCE CODE	63

LIST OF TABLES

Table 0.1: Abbreviations used in this report	xi
Table 1.1: Significance of the study to stakeholders	4
Table 1.2: Operational definition of terms	6
Table 2.1: Illustrative fixture and scheduling view	10
Table 2.2: Comparison of manual and web-based approaches	13
Table 3.1: Development increments	15
Table 3.2: Technologies selected and justification	17
Table 3.3: Functional requirements	17
Table 3.4: Non-functional requirements	18
Table 3.5: Use case description — Record a match result	21
Table 3.6: Conceptual tables and the implemented schema	22
Table 3.7: Development tools and their roles	26
Table 3.8: Security and integrity measures	26
Table 4.1: Development and test environment	28
Table 4.2: Main modules and their implementation	28
Table 4.3: Roles and permissions	34
Table 4.4: Functional test cases and results	45
Table 4.5: Additional integrity and security tests	46
Table 4.6: Requirements traceability matrix	47
Table 4.7: Measured server response times (local environment)	48
Table A.1: users	55
Table A.2: seasons and competitions (events)	55
Table A.3: teams	55
Table A.4: players	55
Table A.5: fixtures (matches and results)	56
Table A.6: match_events and fixture_players	56
Table A.7: Content and communication tables (summary)	57
Table B.1: System test sheet	58
Table D.1: Module specification	60
Table E.1: Acceptance criteria and status	61

LIST OF FIGURES

Figure 3.1: System architecture	20
Figure 3.2: Use case diagram	21
Figure 3.3: Context diagram (level-0 data flow diagram)	22
Figure 3.4: Entity-relationship diagram	24
Figure 3.5: Flowchart of the standings algorithm	25
Figure 4.1: Homepage	30
Figure 4.2: Homepage — sports section	31
Figure 4.3: Fixtures and results with competition, team and month filters	31
Figure 4.4: Standings computed automatically from completed fixtures	32
Figure 4.5: Match centre with score and match events	32
Figure 4.6: Basketball page	33
Figure 4.7: Mobile views — homepage, standings, basketball page and navigation menu	34
Figure 4.8: User registration page	35
Figure 4.9: Login page	35
Figure 4.10: Teams directory	36
Figure 4.11: Squad page with team and position filters	37
Figure 4.12: Player profile with statistics	37
Figure 4.13: Player management in the administration dashboard	38
Figure 4.14: Team management — details, squad and record	38
Figure 4.15: Fixture management in the administration dashboard	39
Figure 4.16: Scheduling conflict detected and rejected	40
Figure 4.17: Result entry and match events	41
Figure 4.18: Administrative dashboard	42
Figure 4.19: Online trial application form	43
Figure 4.20: Trial applications inbox	43
Figure 4.21: News editor	44
Figure 4.22: Published match report	44

LIST OF ABBREVIATIONS

Table 0.1: Abbreviations used in this report

Abbreviation	Meaning
BOUESTI	Bamidele Olumilua University of Education, Science and Technology
CRUD	Create, Read, Update and Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
FR / NFR	Functional / Non-Functional Requirement
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
MVC	Model-View-Controller
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
PHP	PHP: Hypertext Preprocessor
RDBMS	Relational Database Management System
SQL	Structured Query Language
TC	Test Case
UI	User Interface
URL	Uniform Resource Locator
XAMPP	Cross-platform Apache, MariaDB/MySQL, PHP and Perl package
XSS	Cross-Site Scripting

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Sports activities are an important component of educational and organisational development because they provide opportunities for physical exercise, teamwork, discipline, leadership, healthy competition and social interaction. Within schools and universities, activities such as football tournaments, athletics competitions, basketball games and inter-house sports are frequently used to encourage participation and identify talented athletes. Sports participation can therefore contribute to the development of individuals and communities.

Although the sporting activity itself is physical, its administration is fundamentally an information-management task. Before a match takes place, organisers must know who is registered, which team a participant belongs to, what competition is being played, where and when the event will occur, and which officials or organisers are responsible. During the competition, scores and other results must be captured correctly. After the competition, standings and results must be communicated and preserved for reference.

In many small institutions and organisations, these administrative activities are still handled with paper records, spreadsheets, verbal announcements or combinations of these approaches. Such methods can be adequate when an event is very small, but their weaknesses become more visible as the number of teams, participants and fixtures increases. Records may be misplaced, duplicated or entered incorrectly; fixtures can overlap; changes may not reach all participants promptly; and spectators may have difficulty obtaining current results.

Scheduling conflicts, loss of important records, delays in updating match results, difficulties in coordinating participants and human error are recurring problems of manual sports event management. These issues are not merely clerical inconveniences. Incorrect registration information can affect eligibility; a fixture conflict can disrupt an entire tournament schedule; and inaccurate scores can undermine trust in competition results.

Web-based information systems offer a way to centralise these activities. A user can access a web application through an internet-enabled browser, while the application processes requests and stores information in a database. In the sports event management context, this arrangement makes it possible to create a single source of event

information that can be updated by authorised users and accessed by relevant stakeholders.

The project therefore focuses on the design and implementation of a web-based sports event management system capable of supporting athlete registration, team management, event scheduling, score tracking and result reporting. The system follows a client-server model and a three-tier architecture consisting of the presentation, application and database layers (Section 3.9). It is built with HTML, CSS, JavaScript, PHP and MySQL — with the Laravel framework organising the PHP code — using Visual Studio Code and XAMPP as development tools. To evaluate the design on a real case, the system was implemented as *BOUESTI Sports*, the sports portal of Bamidele Olumilua University of Education, Science and Technology (BOUESTI), Ikere-Ekiti, where seven student football clubs (Amapro FC, Elite FC, Young Boys FC, CSC Elites, Engines Boys, Sovereignty FC and AMCOMS) compete internally and basketball is also played on the university's courts.

An important feature of the system is centralisation. Rather than requiring an organiser to maintain separate paper registers, spreadsheets and announcement channels, the platform maintains related records within one database. Registration information is connected to teams, teams to competitions, fixtures to teams, and scores and match events to fixtures. Standings and statistics are then computed from those records. This relational structure supports consistency and makes information easier to retrieve.

The system is also intended to improve communication. Participants can see match schedules and competition results without depending on verbal communication, and spectators obtain published information through the same platform. Administrators update information from a dashboard, reducing the delay between an event and the publication of its official record.

The study is situated within computer science because it applies principles of software engineering, database management, web development, information systems analysis, user interface design and system testing to a practical organisational problem. It is not intended to replace sports officials or the human decisions required in event management. Rather, it provides information-processing support for the repetitive administrative tasks that surround sporting activities.

1.2 Statement of the Problem

Manual management of sports events creates difficulties in maintaining accurate and accessible records. Paper documents can be misplaced or damaged, while spreadsheet files may exist in multiple versions. When several people update records without a common system of control, inconsistencies occur.

Scheduling is another important problem. A sports competition may involve many teams, venues and time slots. If fixture preparation is performed manually, an organiser may accidentally assign the same team to two matches at the same time or fail to communicate a change in schedule. The result can be confusion, delayed matches and unnecessary administrative workload.

Score and result management also require accuracy. Scores must be recorded against the correct fixture and be available to authorised organisers for verification before results are published. Where information is kept in disconnected records, updating one copy does not necessarily update another. This can produce discrepancies between the score sheet, the standings and the public announcement; league tables computed by hand are particularly prone to arithmetic errors.

Communication is affected when there is no centralised digital platform. Participants and spectators may have to ask organisers directly for fixture information or results. This increases the communication burden on organisers and causes information to spread at different times to different people.

Finally, manual approaches make it difficult to produce a consistent historical record of an event. The problem addressed by this project is therefore the absence or inadequacy of an integrated, accessible and reliable web-based mechanism for managing the core administrative activities of small-scale sports events.

1.3 Motivation for the Study

The motivation for the study arises from the gap between the increasing amount of information generated by sports competitions and the limitations of manual administrative processes. Smaller institutions and local organisations often continue to rely on manual methods because accessible and affordable digital solutions are not always available to them.

The project responds by concentrating on essential functions rather than attempting to build an unnecessarily complex professional sports platform. The system provides practical support for registration, team management, scheduling, score recording and result publication. This focus makes the project appropriate for educational institutions

and small organisations.

Another motivation is the educational value of the project. Building the system provides an opportunity to apply computer science concepts to a complete information-system lifecycle: problem identification, requirements analysis, interface design, database modelling, programming, testing and evaluation. The project therefore has both practical and academic value.

1.4 Aim and Objectives

The **aim** of this project is to design and implement a web-based sports event management system that improves the organisation, scheduling and tracking of sports competitions. The specific **objectives** are:

1. To design a user-friendly web-based system for efficient management of sports events, including registration, scheduling and result tracking.
2. To implement a functional platform that enables organisers to manage competitions, teams and scores while allowing users to access event information easily.
3. To evaluate the system's performance, usability and effectiveness in improving the organisation, coordination and transparency of sports event management.

1.5 Research Questions

1. How can a web-based system improve the registration and management of sports participants and teams?
2. How can centralised digital scheduling reduce the likelihood of fixture conflicts and communication delays?
3. How can a database-supported score and result module improve the accuracy and accessibility of sports competition information?
4. How usable and effective is the proposed system for organisers, participants and spectators?
5. To what extent can the proposed system reduce the administrative burden associated with manual sports event management?

1.6 Significance of the Study

Table 1.1: Significance of the study to stakeholders

Stakeholder	Expected significance
Sports organisers	Centralised registration, scheduling, score and result management; standings computed automatically; reduced repetitive administrative work.
Athletes and teams	Easier registration and trial applications; access to fixtures, results and individual statistics.
Spectators	Improved access to published schedules, live and final scores, standings and results on any device.
Educational institutions	Better coordination and documentation of sports activities; a professional digital presence for campus sport.
Researchers and developers	A reference model for applying web technologies to sports event management.

The significance of the study is primarily practical. Organisers require reliable information at different stages of an event. A centralised platform makes the same information available through controlled interfaces, thereby improving coordination.

The study is also significant from a documentation perspective. Digital records can be searched and retained more easily than scattered paper documents. This supports institutional reporting and future reference, provided appropriate data protection and backup procedures are used.

Finally, the study contributes to computer science education by demonstrating how a real-world administrative problem can be translated into requirements, data models, software modules and testable system functions.

1.7 Scope of the Study

The study covers the design and implementation of a web-based sports event management system for an educational institution or small organisation, implemented for the internal competitions of BOUESTI. The core functions are user registration and authentication, athlete and team management, competition (event) creation, fixture scheduling with conflict detection, score and match-event recording, automatic computation of standings and statistics, result publication and administrative management. Supporting functions include online trial applications, contact messages, news, a photo gallery and videos.

The scope includes a responsive web interface for the public, a role-protected administrator dashboard, server-side processing, a relational database and automated system testing focused on functionality, validation, security, data integrity, responsiveness and response time. Football is managed in full; basketball is presented as an information section (programme, courts and how to get involved), with basketball

fixtures and results recommended for a future version.

The project does not cover live video streaming, online payment for participation, or integration with external sports federation databases.

1.8 Limitations of the Study

- Academic time constraints restricted the inclusion of advanced features such as real-time push notifications and automatic fixture generation.
- Development and testing were conducted in a local environment (XAMPP) rather than on large-scale production hosting infrastructure, so large-scale operational performance cannot be claimed without field deployment.
- Verified squad lists were not available in machine-readable form, so the system was tested with a generated sample season of fictional players; a CSV import tool is provided for real data.
- Users require suitable internet connectivity and a web-enabled device.
- The user-satisfaction questionnaire (Appendix C) is designed for administration after deployment; its results are therefore not part of this report.
- The system is intentionally focused on core sports-event administration rather than every possible feature of a professional sports management platform.

1.9 Operational Definition of Terms

Table 1.2: Operational definition of terms

Term	Operational meaning in this study
Web-based system	An application accessed through a web browser over a network.
Sports event	An organised sporting competition (league, cup, friendly or tournament) managed through the platform.
Administrator	An authorised user responsible for configuring competitions, teams, fixtures, scores and results.
Participant	An athlete or team member whose information is managed by the system.
Fixture	A scheduled pairing of a home team and an away team for a particular match.
Score	The recorded number of goals (or points) of each team in a match.
Match event	A recorded incident in a match, such as a goal, assist, card or substitution.
Result	The published outcome of a completed match, shown publicly once the match status is full time.
Standings	The ranking of teams in a competition, computed from completed results.
Database	A structured collection of system information stored and retrieved electronically.
Three-tier architecture	A design separating presentation, application processing and data storage responsibilities.

1.10 Organisation of the Report

The report is organised into five chapters. Chapter One introduces the study and presents the background, problem, motivation, aim, objectives, research questions, significance, scope and limitations. Chapter Two reviews concepts and related work relevant to sports event management, web-based systems, scheduling, databases and software architecture. Chapter Three presents the methodology, requirements analysis and system design. Chapter Four describes the implementation and presents the results of testing and evaluation. Chapter Five summarises the work, draws conclusions, presents recommendations and identifies future extensions. The appendices contain the data dictionary, the completed test sheet, the user questionnaire, module specifications, acceptance criteria, a user and installation guide and selected source code.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews concepts relevant to the design of a web-based sports event management system. Information technology, web-based management systems, software automation and sports administration are the central areas of interest. The review moves from the general concept of sports event management to the information-system functions required to support it, before considering architecture, frameworks, database design, security, usability and the gap that motivates the project.

2.2 Concept of Sports Event Management

Sports event management involves planning, coordinating, administering and documenting the activities required for a sporting competition. Although the exact responsibilities vary according to the sport and size of the event, common activities include participant registration, team formation, scheduling, venue coordination, officiating, score recording, result publication and communication.

From an information-system perspective, sports event management can be viewed as a set of related entities and processes. Participants belong to teams; teams take part in competitions; competitions contain matches; matches have schedules and scores; and completed matches generate results and standings. The relationships between these objects make a structured database useful.

An effective management platform should therefore support both data capture and information presentation. It is not sufficient to store registration data if organisers cannot retrieve it quickly. Similarly, recording scores has limited value if participants cannot access verified results. The system must connect administrative functions with user-facing information.

2.3 Sports Event Administration

Sports administration begins before competition day. Organisers identify the sport, define the competition structure, collect entries and verify participants. Registration data forms the foundation for later processes because fixtures and results depend on knowing which teams or individuals are eligible.

During competition, administration becomes time-sensitive. Organisers need to know which match is next, where it will take place, which teams are involved and whether previous results have been recorded. After matches, scores must be entered and verified, and the appropriate result made available to users.

These activities demonstrate why a centralised system is valuable. Each process produces information that is consumed by another process: a team registered in the system later appears in a fixture; that fixture later receives a score; and the score becomes part of the published results and standings.

2.4 Information Systems in Event Management

Information systems combine people, processes, data and technology to support organisational activities (Laudon and Laudon, 2020). Web-based management systems have been adopted in many sectors to automate administrative processes and improve data accessibility.

Automation is particularly useful for repetitive operations. A registration form can validate required fields before data are stored. A scheduling module can retrieve available teams and competition information from the database and check for conflicts. A result module can associate a score with an existing fixture rather than requiring an organiser to rewrite all fixture details. A well-designed system captures each fact once, at its source, and derives other information — such as standings — automatically.

Pressman and Maxim (2020) describe software as a means of automating complex tasks, reducing human error and improving organisational efficiency, while Sommerville (2016) emphasises the need for maintainable and scalable applications that are understood in relation to the organisation that uses them.

2.5 Web-Based Information Systems

A web-based information system provides functionality through a browser-based interface over the HyperText Transfer Protocol, whose architectural principles were formalised by Fielding (2000). The major advantage in the context of this study is accessibility: authorised users can interact with the application without installing software on every device.

A typical web system separates presentation from processing and data storage. The browser presents forms and pages, the server handles business rules and authentication, and the database stores persistent information. This separation supports maintainability because a change to one layer does not necessarily require redesigning every other

layer.

Web systems also support centralised updates. When an administrator publishes a new result, the updated information is retrieved from the same database by every user accessing the relevant page. This is preferable to maintaining multiple independent copies of a result list.

2.6 Sports Registration and Team Management

Registration is the process through which participant and team information enters the system. A useful registration module captures enough information to identify the participant or team while avoiding unnecessary duplication. Validation is important because incorrect email addresses, empty names or duplicated identifiers make later processes unreliable.

Team management extends registration by associating athletes with teams and teams with competitions. The database should represent these relationships explicitly, allowing the application to answer questions such as which players belong to a team or which teams are taking part in a competition.

Role-based access is also relevant. An administrator may create or modify team information, while an ordinary participant may only view information relevant to the event. Separating permissions reduces the risk of unauthorised modification.

2.7 Scheduling and Fixture Management

Scheduling is one of the most coordination-intensive functions in a sports event. A fixture must identify the participating teams, date, time, venue and competition stage. The schedule must also avoid obvious conflicts within the constraints of the event.

Automated scheduling does not eliminate the need for human judgement. Instead, the system stores and displays the schedule consistently and applies basic validation rules — for example, preventing a team from being assigned to two fixtures on the same day, or a team from being drawn against itself.

Fixture management should also support updates. If a match is postponed or a time changes, the administrator should be able to update the central record so that subsequent users retrieve the new information rather than an outdated copy. Table 2.1 illustrates the kind of schedule a fixture module maintains.

Table 2.1: Illustrative fixture and scheduling view

Match	Home team	Away team	Date	Time	Status
M01	Team Alpha	Team Beta	Day 1	10:00	Scheduled
M02	Team Gamma	Team Delta	Day 1	12:00	Scheduled
M03	Winner M01	Winner M02	Day 2	14:00	Pending

2.8 Score and Result Management

Score management requires a clear link between a score and the match to which it belongs. A result should not exist independently of a fixture because that creates ambiguity. The database design should therefore identify matches with unique identifiers and associate scores with those identifiers.

Result publication is the point at which internal administrative data becomes information for participants and spectators. The system should distinguish between an editable administrative record and a published result. This separation reduces accidental public display of incomplete information.

Most association football leagues rank teams by awarding three points for a win, one for a draw and none for a defeat, breaking ties by goal difference and then goals scored. Because standings are a pure function of the completed results, they should be computed from the results rather than stored and edited by hand; this removes a whole class of inconsistency errors while preserving the integrity of previously recorded results.

2.9 Database Management

The relational model, introduced by Codd (1970), represents data as tables related through primary and foreign keys. It is suitable for this project because the core data have clear relationships: users, teams, players, competitions, matches and match events. Normalisation reduces repetition and update anomalies (Elmasri and Navathe, 2016; Connolly and Begg, 2015).

Database integrity is important in sports management. A match should refer to valid teams, and a match event should refer to a valid match and a player of one of its teams. Constraints and application-level validation prevent orphan records, and unique identifiers reduce accidental duplication.

MySQL (and its compatible fork MariaDB, bundled with XAMPP) is appropriate for a conventional web application because it supports relational tables, structured queries, indexes, constraints and transactions needed for common CRUD operations.

2.10 Three-Tier Architecture and the MVC Pattern

A three-tier architecture separates the presentation layer, the application layer and the database layer. The presentation layer is the user-facing component built with HTML, CSS and JavaScript. The application layer handles requests, authentication, scheduling logic, score updates and communication with the database. The database layer stores persistent information.

Separating these concerns has several advantages. The user interface can evolve without changing the underlying data model; business rules are centralised on the server; and database access is controlled rather than exposed directly to the browser.

Within the application layer, the Model-View-Controller (MVC) pattern, first described for Smalltalk-80 user interfaces (Krasner and Pope, 1988), separates the **Model** (data and business rules), the **View** (presentation) and the **Controller** (handling input and coordinating the model and view). This separation improves maintainability and testability (Gamma et al., 1994; Fowler, 2002).

2.11 Web Development Frameworks

A web framework provides the common infrastructure required by most web applications — routing, request handling, database access, templating, authentication, validation and security protections — so that developers can concentrate on application-specific logic. Laravel is an open-source PHP framework that follows the MVC pattern and provides the Eloquent object-relational mapper, database migrations, the Blade templating engine, form-request validation, middleware, authentication scaffolding and built-in support for automated testing (Laravel, 2025). Using a mature framework reduces the amount of security-sensitive code that must be written by hand.

2.12 Security, Privacy and Access Control

Sports systems contain personal information such as participant names and contact details. Security should therefore be considered even when the system is designed for a small institution. The OWASP Top 10 (OWASP Foundation, 2021) lists the most critical web application risks, including broken access control, injection and cross-site scripting.

Authentication ensures that only registered users access protected functions, while authorisation distinguishes administrative functions from ordinary viewing functions. Passwords should never be stored as plain text; they should be hashed. Server-side validation should be applied even when client-side validation exists. Database queries

should use parameter binding to prevent injection, output should be escaped, forms should carry anti-forgery tokens, and sessions should be managed carefully. Backup procedures are also important because a centralised database becomes a critical source of institutional information.

2.13 Usability in Sports Event Systems

Usability refers to how easily intended users can understand and operate the system (Nielsen, 1993). Sports organisers may need to perform tasks quickly while an event is active, so interfaces should use clear labels, predictable navigation, concise forms and clear error messages.

Participants and spectators have different needs. Administrators require management functions, while spectators mainly need schedules, scores and results. A well-designed system should not expose administrative controls to ordinary users.

Responsive web design (Marcotte, 2010) is also relevant because users access schedules and results from phones, tablets and desktop computers. Accessibility guidelines such as WCAG 2.1 (W3C, 2018) add criteria such as keyboard operability, visible focus and text alternatives for images.

2.14 Review of Related Approaches

Digital systems for athlete registration, match scheduling, score tracking and result publication are widely used, from official websites of professional competitions to subscription-based platforms for amateur leagues. For a small institution, however, general solutions may involve subscription costs, may not be branded to the institution, may not model student-specific information, and may store participant data with third parties. Table 2.2 compares the main approaches functionally.

Table 2.2: Comparison of manual and web-based approaches

Function	Manual approach	Web-based approach (this project)
Registration	Paper forms / spreadsheets	Centralised electronic registration and trial applications
Team management	Separate lists	Linked team and player records
Scheduling	Manually prepared fixtures	Central fixture records with conflict validation
Score tracking	Paper score sheets / announcements	Database-linked scores and match events
Standings	Computed by hand	Computed automatically from completed results
Result publication	Notice boards / verbal updates	Web-accessible published results

Function	Manual approach	Web-based approach (this project)
Documentation	Scattered records	Searchable structured database preserved across seasons

2.15 Theoretical Perspective

The study can be understood through the systems-development perspective, in which a problem domain is analysed, requirements are identified, and a solution is designed, implemented and evaluated. Software engineering principles provide a further foundation: requirements should be traceable to implemented functions, components should have clear responsibilities, and testing should provide evidence that functions behave as intended (Sommerville, 2016; Beck, 2002).

From an information-management perspective, the project emphasises the transformation of raw sports-event data into useful information. Registration entries, fixtures, scores and match events are raw records; schedules, standings, player statistics and published results are organised outputs that support decision-making and communication.

2.16 Gap in Literature and Rationale for the Study

The central gap is practical rather than a claim that no sports-management systems exist. Smaller institutions and local organisations still depend on manual processes and lack accessible, affordable digital solutions that fit their structure.

The project therefore targets simplicity and integration. Instead of focusing on advanced professional features, it concentrates on a coherent set of core functions that can be deployed in an educational or small-organisation context. The value of the work lies in bringing registration, scheduling, score management, standings and result publication into one institution-owned platform.

CHAPTER THREE

METHODOLOGY AND SYSTEM ANALYSIS

3.1 Research Design

The study adopts a system development research design: the problem domain is analysed, requirements are identified, and a web-based information system is designed, implemented and evaluated against those requirements. This chapter presents the development approach, the methods used to gather requirements, the analysis of the existing process, the requirements of the new system and its design — architecture, use cases, data flow, database, algorithms, interfaces, security and testing strategy.

3.2 Development Approach

An **iterative and incremental** development approach was adopted (Pressman and Maxim, 2020; Sommerville, 2016). The project has identifiable modules with dependencies: teams must exist before fixtures are created, fixtures must exist before scores are attached, and results and standings are produced from valid scores. Each increment was designed, implemented, tested and reviewed before the next began, and feedback from testing was used to correct defects and improve usability (Table 3.1).

Table 3.1: Development increments

Increment	Deliverables
1. User interface	Responsive public website design converted into reusable Blade layouts and components.
2. Data model	Relational schema (migrations), Eloquent models and relationships for seasons, competitions, teams and players.
3. Fixtures and results	Fixture scheduling, result entry, match events, line-ups; league table and statistics services.
4. Administration	Role-protected admin dashboard with create, view, update and delete screens for every entity.
5. Public information	Public pages connected to the database: fixtures, results, standings, teams, squads, player profiles, news and media.
6. Registration and communication	User accounts, trial applications, contact messages and administrative inboxes.
7. Supporting features	Rich-text news editor with sanitisation, image optimisation, CSV import, sample-season generator, basketball and facilities pages, fixture conflict checking.
Continuous	Automated tests written alongside each increment and re-run after every change.

3.3 Methods of Data Collection

Requirements were gathered through the following methods:

- **Observation** of how sports information is currently handled — fixtures and results announced on notice boards and in messaging groups, tables computed by hand.
- **Informal discussions** with students involved in the internal teams about the information they need and the difficulties they face.
- **Review** of existing sports-competition websites and of the literature in Chapter Two to identify the standard functions of fixture, result and standings management.
- **A structured questionnaire** (Appendix C) was designed to measure user satisfaction after deployment. It had not been administered at the time of writing; its results should be added once it has been completed by representative users.

3.4 Analysis of the Existing System

In the existing process, registration of players occurs through paper lists or spreadsheets kept by team officials; fixtures are agreed among organisers and announced on notice boards or in messaging groups; scores are reported verbally or by message after each match; any league table is computed by hand; and records of scorers and cards, if kept at all, are held privately by individual organisers.

3.4.1 Weaknesses of the existing system

- **Fragmentation:** each activity produces information that another needs, but there is no shared store, so data are re-entered and become inconsistent.
- **Scheduling conflicts:** nothing prevents a team from being given two matches on the same day, and changes do not reach everyone.
- **Error-prone standings:** hand-computed tables are slow to update and prone to arithmetic and ordering errors.
- **Poor accessibility:** information in chat groups is buried quickly and unavailable to students outside the groups.
- **Loss of history:** results and statistics of previous seasons are not preserved.
- **No formal channels** for registration, trial applications or enquiries, and no control over who sees participants' personal data.

3.5 The Proposed System

The proposed system is a web-based platform with two interfaces sharing one database: a **public website** for participants and spectators, and an **administration dashboard** for authorised organisers. Organisers record each fact once — a team, a player, a fixture, a

score, a goal — and the system derives everything else: schedules, standings, player and team statistics, the match centre and the homepage summaries. The database becomes the controlled source of event information, while the interfaces provide role-appropriate access.

Table 3.2: Technologies selected and justification

Layer	Technology	Justification
Presentation	HTML5, CSS3, JavaScript; Blade templates; Tailwind CSS 3 and Alpine.js 3 (admin)	Standard web technologies; responsive layouts; lightweight for mobile data connections.
Application	PHP 8.2 with the Laravel 12 framework	MVC structure, routing, validation, authentication, CSRF protection and testing tools out of the box; widely hosted.
Database	MySQL / MariaDB 10.4 (SQLite in-memory for tests)	Reliable open-source RDBMS with foreign keys and transactions; bundled with XAMPP.
Authentication	Laravel Breeze	Secure registration, login, password reset and password hashing.
Development	Visual Studio Code, XAMPP, Composer, Node.js / Vite	Free, widely used local development and build tools.
Testing	Pest 3 (on PHPUnit); headless Microsoft Edge	Automated feature tests; automated responsive checks.

3.6 Functional Requirements

Table 3.3: Functional requirements

ID	Requirement	Description
FR01	User registration	The system shall allow users to create accounts with a name, a valid unique email and a confirmed password, and students to apply for team trials.
FR02	Authentication	The system shall authenticate users before protected operations and allow them to log out.
FR03	Team management	The administrator shall create, view, update and archive teams (name, short name, colours, logo, coach, captain).
FR04	Player management	The system shall record athletes and associate each with a team; shirt numbers shall be unique within a team.
FR05	Event management	The administrator shall create and manage seasons and competitions (league, cup, friendly, tournament).
FR06	Fixture management	The administrator shall create and update fixtures; a team shall not play itself or be scheduled for two active fixtures on the same day.
FR07	Score recording	Authorised users shall record scores against valid fixtures, together with match events (goals, assists, cards, substitutions) and line-ups restricted to the fixture's teams.
FR08	Result publication	A result shall become public when the fixture is marked full time; the league table shall be computed automatically from completed results.

ID	Requirement	Description
FR09	Information access	Participants and spectators shall view schedules, results, standings, teams, squads, player profiles, news and media without logging in.
FR10	Reporting	The system shall provide organised views: fixtures filtered by competition, team and month; a match centre; standings with form guide; player and team statistics; an administrative dashboard.
FR11	Communication	The system shall accept contact messages and trial applications and provide administrative inboxes with status tracking.
FR12	Content	The administrator shall publish news (rich text), photos and videos.
FR13	Data import	The system shall import teams, players, staff, fixtures and events from CSV files, rejecting the whole import if any row is invalid.

3.7 Non-Functional Requirements

Table 3.4: Non-functional requirements

Category	Requirement
Usability	Interfaces shall be clear, consistent and easy to navigate, with labelled forms and specific error messages beside each field.
Performance	Common pages shall return within one second under the intended small-organisation workload; statistics shall use aggregate queries; images shall be optimised.
Security	Protected functions shall require authentication and the administrator role; all input shall be validated on the server; output shall be escaped.
Reliability	Valid records shall be preserved; deleting teams, players or news shall not destroy historical match records.
Maintainability	Code shall follow MVC with services for business logic, form requests for validation and reusable view components.
Compatibility	The application shall work in modern browsers (Chrome, Edge, Firefox, Safari).
Responsiveness	Pages shall be usable on screens from 320 px to 1440 px wide without horizontal scrolling.
Scalability	Additional seasons, competitions, teams and records shall be added through data, without redesigning the application.
Privacy	Matriculation numbers, state of origin and applicants' contact details shall never appear on public pages.
Availability	Users shall be able to access the platform whenever the hosting environment and network are available.

3.8 Feasibility Analysis

3.8.1 Technical feasibility

The technology stack is mature and widely documented. HTML, CSS and JavaScript provide the browser interface, PHP with Laravel provides server-side processing, and MySQL stores relational data. Visual Studio Code and XAMPP provide a complete

local development environment, and the finished system runs on any host supporting PHP 8.2 and MySQL, including low-cost shared hosting.

3.8.2 Operational feasibility

The system's functions correspond closely to tasks organisers already perform, so the main operational change is how information is recorded and retrieved. Training focuses on the administrator workflow (Appendix F); spectators need no training because the public site requires no login.

3.8.3 Economic feasibility

All development tools and frameworks used are free and open source. The running cost is limited to hosting and a domain name, which is low for an educational institution.

3.8.4 Schedule feasibility

Academic time constraints were managed by prioritising the essential modules — authentication, registration, teams, competitions, fixtures, scores and results — and adding supporting features in later increments.

3.9 System Architecture

The system follows a client-server, three-tier architecture implemented with Laravel's MVC structure (Figure 3.1). The browser forms the presentation tier; the Laravel application forms the application tier; and MariaDB/MySQL forms the data tier. The request flow is:

1. The user opens a page or submits a form in the browser.
2. The request reaches a route; middleware applies authentication, the administrator check, CSRF verification and rate limiting.
3. A form request validates the input (for example, the fixture conflict rule).
4. A controller coordinates the request, delegating business logic such as standings to a service class.
5. Eloquent models read from or write to the database using parameterised queries.
6. The database returns the requested records or confirms the operation.
7. A Blade view renders the schedule, score, result or confirmation, which is returned to the browser.

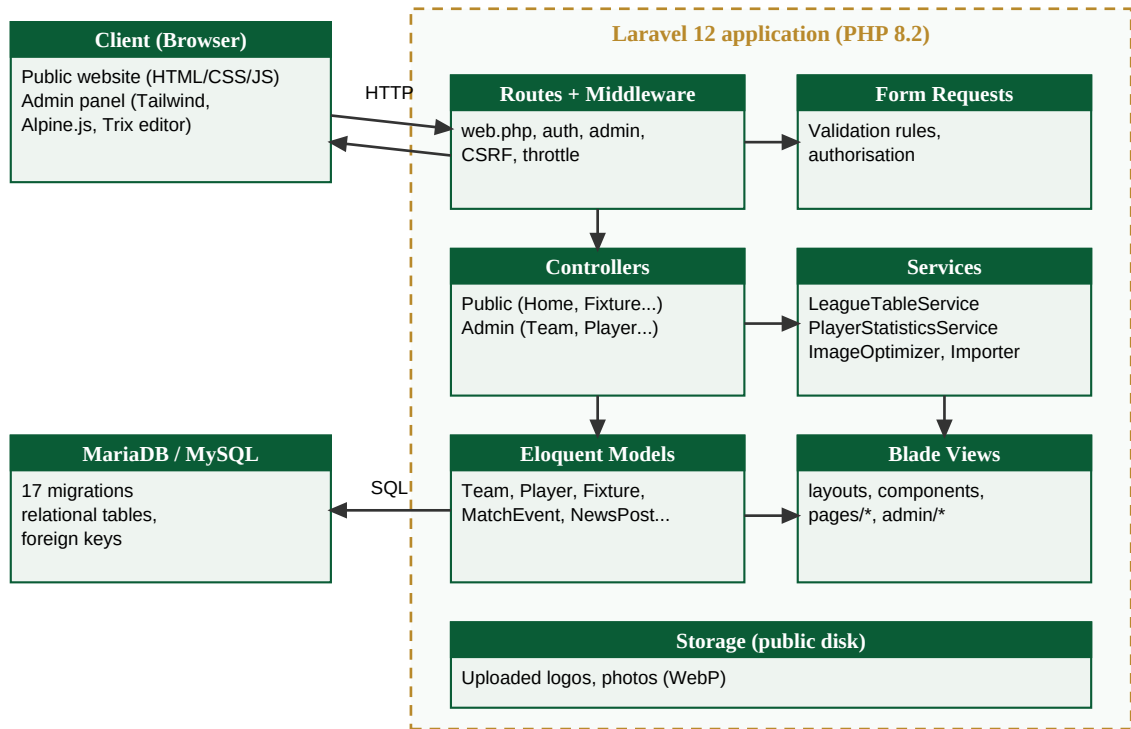


Figure 3.1: System architecture

3.10 Use Case Model

Three actors interact with the system (Figure 3.2). A **Visitor** (spectator) browses public information. A **Student** (participant) is a visitor who registers an account, applies for trials or sends a message. An **Administrator** is an authenticated organiser with the admin role who manages all data. Table 3.5 describes the principal use case.

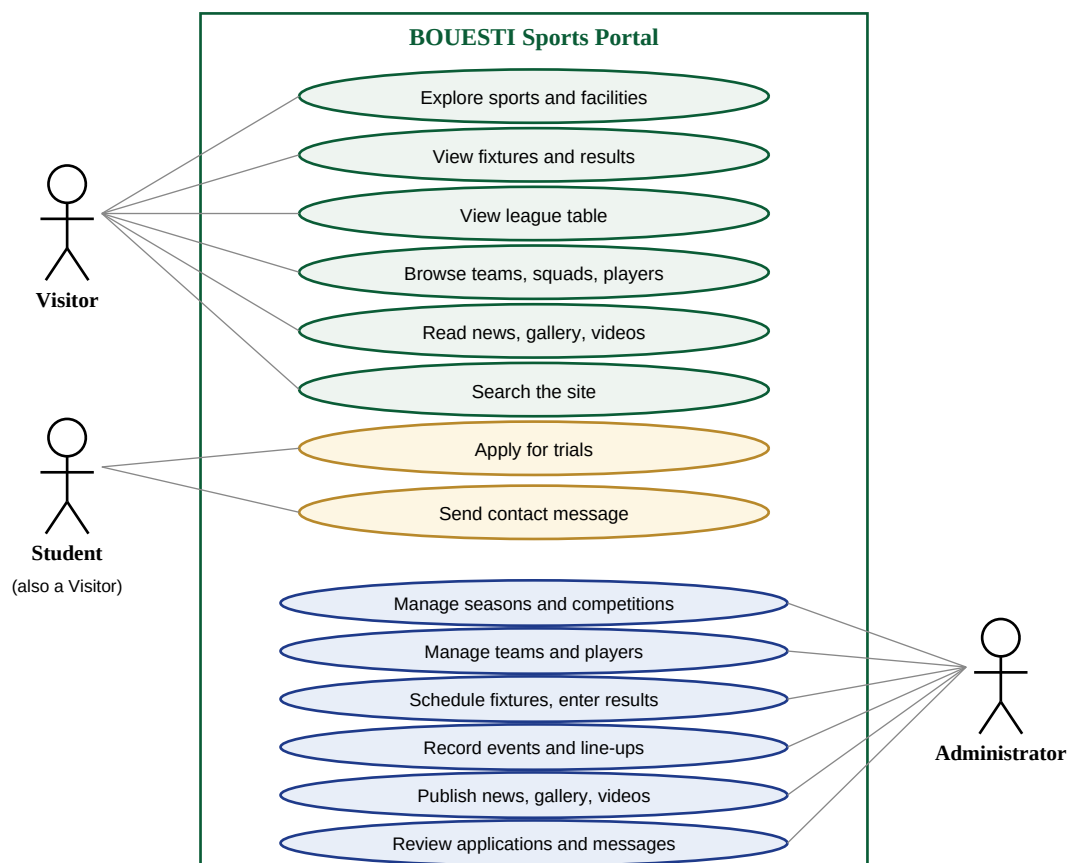


Figure 3.2: Use case diagram

Table 3.5: Use case description — Record a match result

Item	Description
Actor	Administrator
Pre-condition	The administrator is logged in; the fixture exists.
Main flow	1. Open Fixtures and select the match. 2. Choose status “Full time” and enter both scores. 3. Optionally add referee, attendance and a summary. 4. Save. 5. Add goal, assist, card and substitution events; for each, choose the team — the player list shows only that team's players. 6. Tick each team's line-up.
Alternative flow	A score is missing for a completed match: a validation message is shown and nothing is saved. A player not in the selected team: the event is rejected.
Post-condition	The public result, league table, team and player statistics, homepage and match centre reflect the result immediately.

3.11 Data Flow Design

Figure 3.3 shows the context-level (level-0) data flow diagram. Visitors and students send requests, registrations, trial applications and contact messages, and receive published pages; administrators supply competition data and content and receive the dashboard, statistics and inboxes; the system can notify the organisers by email when a

contact message arrives. Because all flows pass through one database, no separate copies of the same information are maintained.

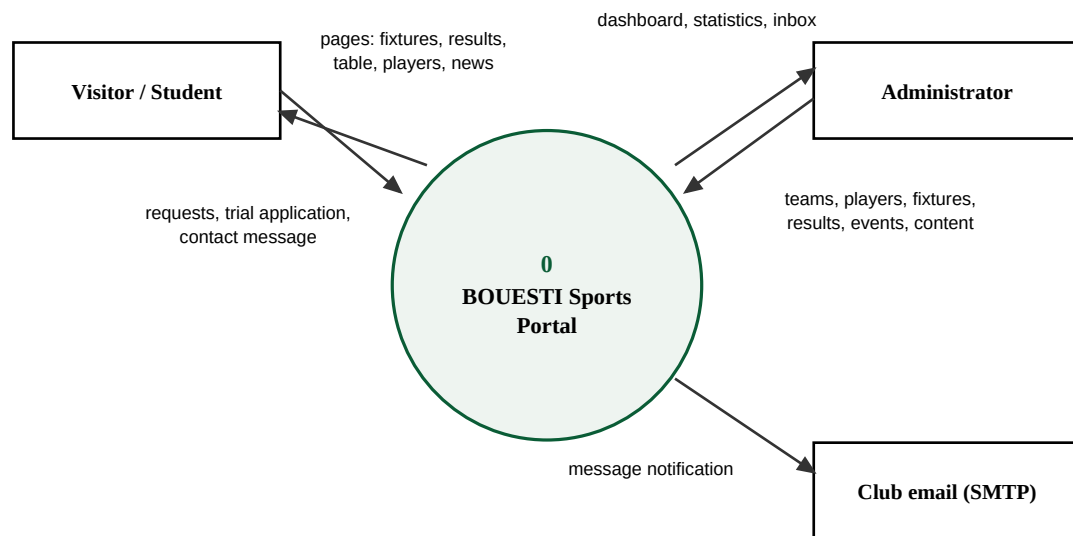


Figure 3.3: Context diagram (level-0 data flow diagram)

3.12 Database Design

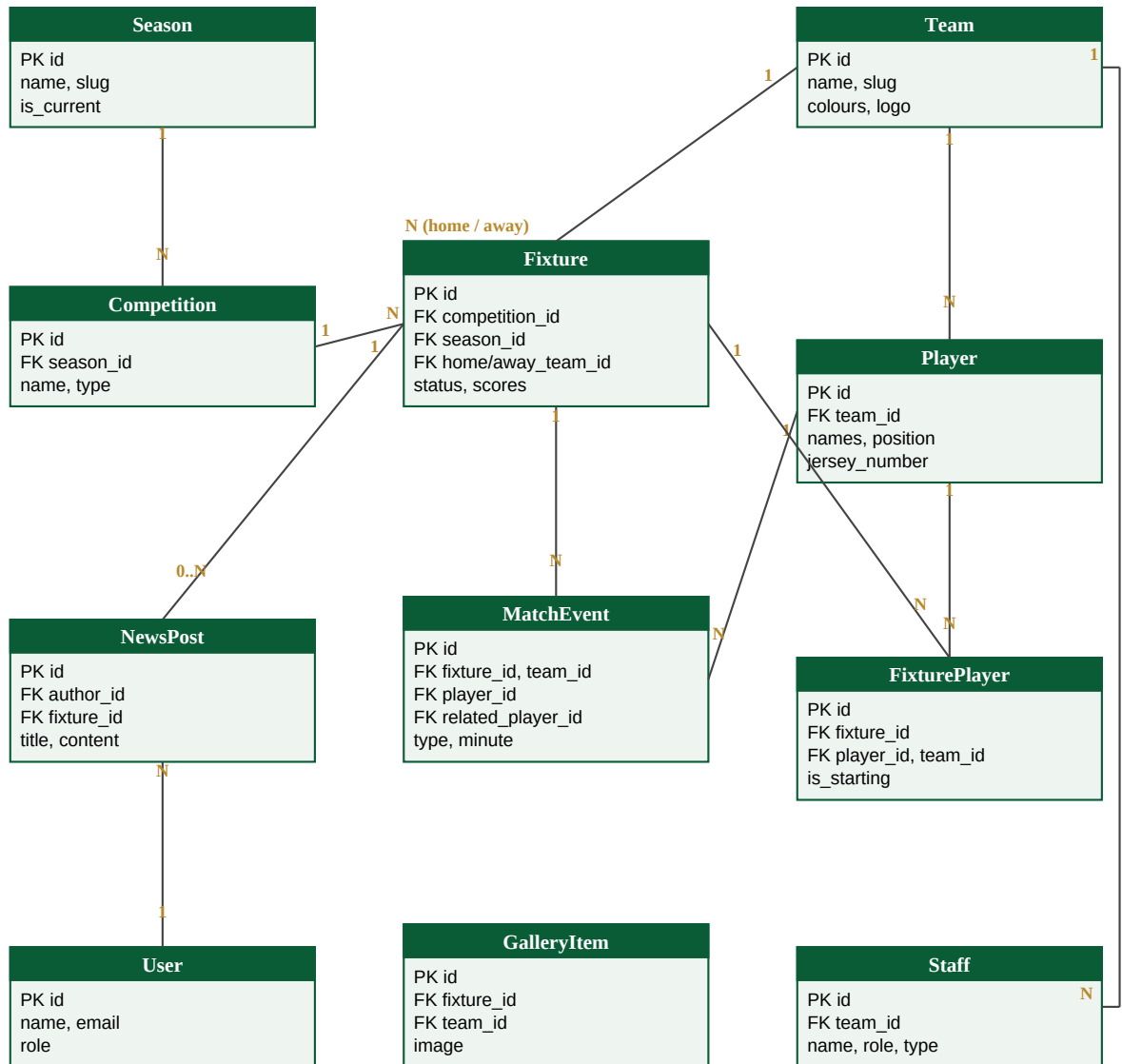
The database was designed in third normal form. Each real-world entity has its own table with a surrogate primary key, and relationships are enforced by foreign keys. The conceptual tables identified during analysis map onto the implemented schema as shown in Table 3.6. Two design decisions are significant. First, the general “event” is modelled as a **season** containing one or more **competitions**, so the same structure supports leagues, cups, friendlies and tournaments. Second, results and standings are **not stored separately**: a fixture's status and scores are its result, and the standings are computed from completed fixtures, which removes update anomalies. Figure 3.4 shows the entity-relationship diagram; the full data dictionary is in Appendix A.

Table 3.6: Conceptual tables and the implemented schema

Conceptual table	Implemented as	Notes
users	users	name, email, hashed password, role (admin user).
teams	teams	Soft-deleted so past results keep their names.
players	players	team_id foreign key; position, shirt number; private fields hidden from the public.
events	seasons, competitions	A season (e.g. 2026/2027) contains competitions of type league, cup, friendly or tournament.

Conceptual table	Implemented as	Notes
matches	fixtures	home and away team, date, kick-off, venue, status (scheduled, live, completed, postponed, cancelled), scores.
results	fixtures.status + scores; computed standings	A result is public once status is “completed”; standings are derived, never typed.
(new)	match_events, fixture_players	Goals, assists, cards and substitutions; line-ups. Source of player statistics.
(new)	news_posts, gallery_items, videos, staff, trial_applications, contact_messages	Content, people and communication.

Deletion rules were chosen deliberately: seasons, competitions and teams referenced by fixtures are protected (*restrict*); match events and line-ups are removed with their fixture (*cascade*); optional links such as a photo's fixture are cleared (*set null*); and teams, players, news and staff are soft-deleted so that historical results remain correct.



GalleryItem optionally references Fixture and Team. Stand-alone: videos, trial_applications, contact_messages.

Figure 3.4: Entity-relationship diagram

3.13 Algorithm Design

3.13.1 Standings computation

The standings are computed on demand by the *LeagueTableService* (Figure 3.5). Every team drawn in the competition receives a row. Completed fixtures are processed in date order: both teams' played, goals for and goals against are updated; the result adds a win, draw or loss and 3, 1 or 0 points; and the result is appended to the team's form. Rows

are sorted by points, goal difference, goals for and name. Because the table is a function of the results, entering or correcting any result is reflected immediately and consistently.

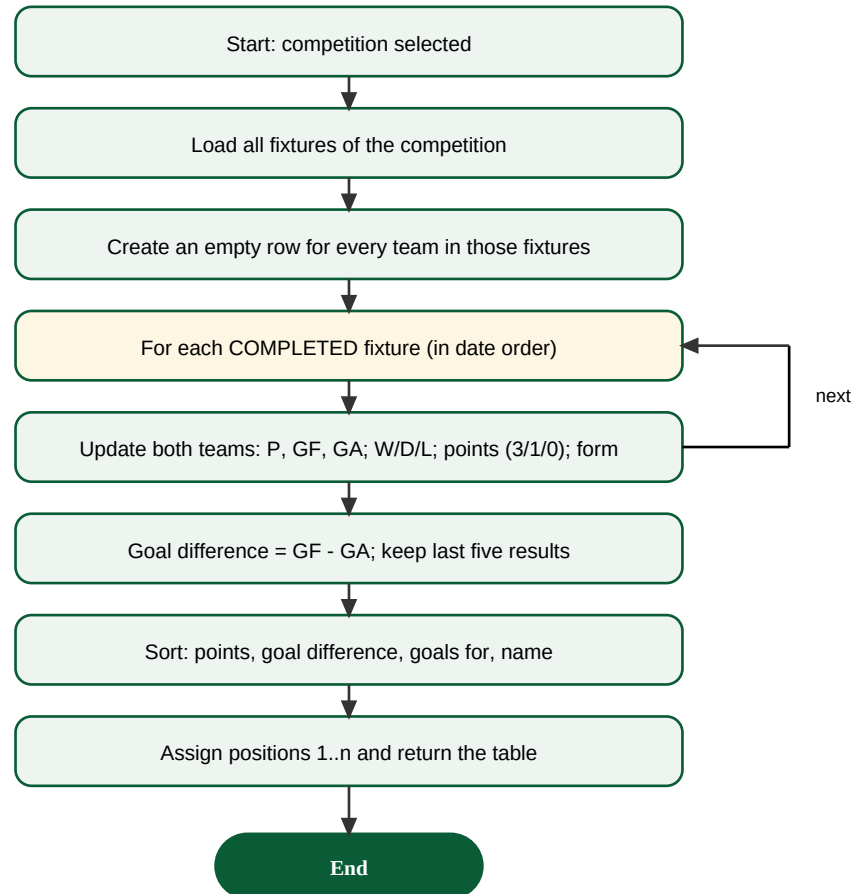


Figure 3.5: Flowchart of the standings algorithm

3.13.2 Fixture conflict detection

When a fixture is created or edited, after the normal field validation passes, the system searches for another fixture on the same date, involving either of the two teams, whose status is not postponed or cancelled (the fixture being edited is excluded). If one exists, the save is rejected with a message naming the clashing match. A team therefore cannot be scheduled for two active matches on the same day, and postponing a match frees the day for rescheduling.

3.13.3 Player statistics

The *PlayerStatisticsService* derives each player's appearances (line-up or event involvement), goals, assists, yellow and red cards and, for goalkeepers, clean sheets, from live and completed matches only, using aggregate SQL queries rather than loading

records into memory.

3.14 Input and Output Design

Inputs are captured through validated forms: registration and login forms; administrative forms for every entity, with dropdowns restricted to valid options, date and time pickers and image inputs limited to JPG, PNG and WebP; the result, event and line-up forms on the match page; the public trial and contact forms, protected by a hidden honeypot field and rate limiting; and CSV files for bulk import. Validation errors are displayed beside the relevant field with an icon and message, not colour alone.

Outputs are the public pages — homepage, fixtures and results, match centre, standings with form guide and top scorers, teams, squads, player profiles, news, gallery, videos, staff, basketball and facilities pages — and the administrative dashboard with counts, upcoming fixtures, recent results and inbox alerts.

3.15 Development Tools and Technologies

Table 3.7: Development tools and their roles

Tool / technology	Role
HTML / Blade	Structure of web pages and forms; reusable layouts and components
CSS / Tailwind CSS	Presentation, layout and responsive styling
JavaScript / Alpine.js	Client-side interaction: menus, filters, countdown, dynamic player lists
PHP 8.2 / Laravel 12	Server-side application processing (MVC)
MySQL / MariaDB	Relational database storage
Visual Studio Code	Source-code editor
XAMPP	Local Apache, MariaDB and PHP environment
Composer / npm / Vite	Dependency management and production asset building
Pest (PHPUnit)	Automated testing
Chrome / Edge / Firefox	Browser testing and user access

3.16 Security and Data Integrity Design

Table 3.8: Security and integrity measures

Threat / risk	Measure
Unauthorised administrative access	Authentication plus an <i>admin</i> middleware that checks the user's role on every admin route; form requests re-check authorisation; the role cannot be mass-assigned.

Threat / risk	Measure
Password compromise	Passwords hashed with bcrypt; no default administrator password outside the local environment.
SQL injection	All database access through Eloquent / the query builder with bound parameters.
Cross-site scripting	Blade escapes output by default; rich text is cleaned by an allow-list HTML sanitiser.
Cross-site request forgery	CSRF token on every state-changing form.
Invalid or tampered data	Server-side validation of every field and relationship: a match event's team must be in the fixture and its player must belong to that team; schedule conflicts are rejected.
Orphan or lost records	Foreign keys with restrict / cascade / set-null rules; soft deletes for teams, players, news and staff.
Malicious uploads	Only images within size limits; files re-encoded to WebP under random names, stripping metadata.
Spam and abuse	Honeypot field and rate limiting (5 submissions per minute) on public forms.
Privacy	Matriculation numbers, state of origin and applicants' contact details are admin-only.

3.17 Testing Strategy

Testing was performed at unit, module (feature) and integration levels. Automated feature tests drive the application through HTTP requests exactly as a browser would — submitting forms, following redirects and inspecting rendered pages — against a fresh in-memory database for each test, so every test starts from a known state. Each functional requirement is traced to one or more test cases (Section 4.14). Registration was tested with valid and invalid input; team management for creation and listing; fixture management for correct team assignment, conflicts and updates; and score management by entering scores and verifying the correct fixture, standings and statistics. Responsiveness was tested automatically in a headless browser at ten screen widths, and response times of common pages were measured. The user questionnaire (Appendix C) is reserved for post-deployment usability evaluation.

CHAPTER FOUR

SYSTEM IMPLEMENTATION AND EVALUATION

4.1 Introduction

This chapter describes the implemented system and the results of its evaluation. The system was implemented as *BOUESTI Sports*, the sports portal of Bamidele Olumilua University of Education, Science and Technology, Ikere-Ekiti. The screenshots were captured from the running system populated with a generated sample season (Section 4.4); all test results and measurements reported were obtained by executing the implemented system.

4.2 Implementation Environment

Table 4.1: Development and test environment

Component	Specification
Hardware	Intel Core i7-8665U (1.90 GHz), 16 GB RAM
Operating system	Microsoft Windows 11
Local server stack	XAMPP (Apache, MariaDB 10.4.32) and PHP's built-in development server
Language / framework	PHP 8.2.12; Laravel Framework 12
Front-end tooling	Node.js 22, Vite 7, Tailwind CSS 3.4, Alpine.js 3, Trix 2
Testing	Pest 3 on PHPUnit (SQLite in-memory database); headless Microsoft Edge for responsive checks
Editor	Visual Studio Code
Client browsers	Chrome, Edge and Firefox on desktop; mobile emulation for phones and tablets

4.3 System Structure

The implemented system comprises 14 Eloquent models, 38 controller classes (public and admin), 17 database migrations, 116 Blade view files, 6 service classes and 121 named routes, of which 71 belong to the administration dashboard; application PHP code amounts to about 5,400 lines. The source follows Laravel's standard folder structure, which separates presentation (resources/views), application logic (app/Http, app/Services), data (app/Models, database/migrations) and configuration.

Table 4.2: Main modules and their implementation

Module	Key components	Function
Authentication	Laravel Breeze controllers, EnsureUserIsAdmin middleware, users.role	Registration, login, logout, password reset; restriction of the admin dashboard.
Team and athlete management	Admin\TeamController, Admin\PlayerController, TeamRequest, PlayerRequest	Teams, squads, player records and statistics.
Event and fixture management	Admin\SeasonController, Admin\CompetitionController, Admin\FixtureController, FixtureRequest	Seasons, competitions and fixtures with conflict checking.
Score and result management	FixtureResultRequest, Admin\MatchEventController, Admin\LineupController	Scores, match events, line-ups and result publication.
Standings and statistics	LeagueTableService, TeamStatisticsService, PlayerStatisticsService	Derives standings and statistics from results and events.
Public information	HomeController, FixtureController, LeagueTableController, TeamController, PlayerController, NewsController, FrontendController	Schedules, results, standings, teams, players, news, basketball and facilities.
Communication	TrialApplicationController, ContactController and admin inboxes	Trial applications and contact messages.
Content and media	NewsController (admin), RichText sanitiser, ImageOptimizer	News, gallery and videos.
Data loading	FootballImporter (football:import command), SampleDataSeeder	CSV import of real data; generation of a sample season.

4.4 Database Implementation and Test Data

The database was created with 17 Laravel migrations that define every table, column type, index, unique constraint and foreign key, so the same schema can be recreated on any server with one command. Seeders create the current season, the league competition, the seven internal teams and the administrator account.

Because verified squad lists were not available, a deterministic *SampleDataSeeder* generates a realistic season for testing and demonstration: 126 fictional players (18 per team), 22 fictional staff, and a double round-robin league of 42 fixtures in which matches already due are played with consistent scores, goals, assists, cards, substitutions and line-ups. Real data can replace it through the *football:import* command (Appendix F).

4.5 User Interface Implementation

The public interface uses a consistent navigation bar with grouped menus (Sports, Club, Team, Fixtures, News, Media, Fans) and a site-wide search. The homepage (Figure 4.1) presents the next fixture with a live countdown, recent results, featured players, news and the university's sports and facilities (Figure 4.2). Fixtures can be filtered by

competition, team and month (Figure 4.3); the standings page shows the computed table with each team's form (Figure 4.4); and the match centre shows the score, minute-by-minute events and line-ups (Figure 4.5). The Basketball page (Figure 4.6) presents the university's second sport. Every page adapts to phone screens (Figure 4.7).



Figure 4.1: Homepage

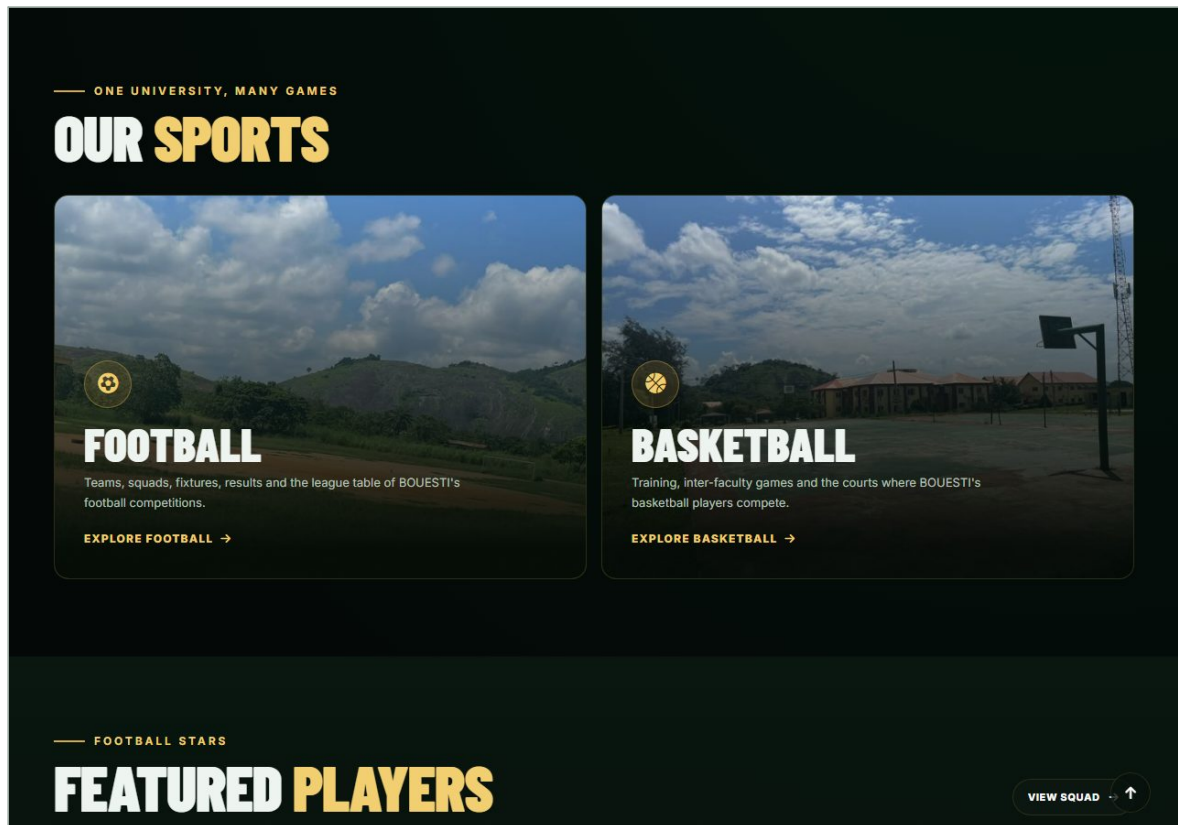


Figure 4.2: Homepage — sports section

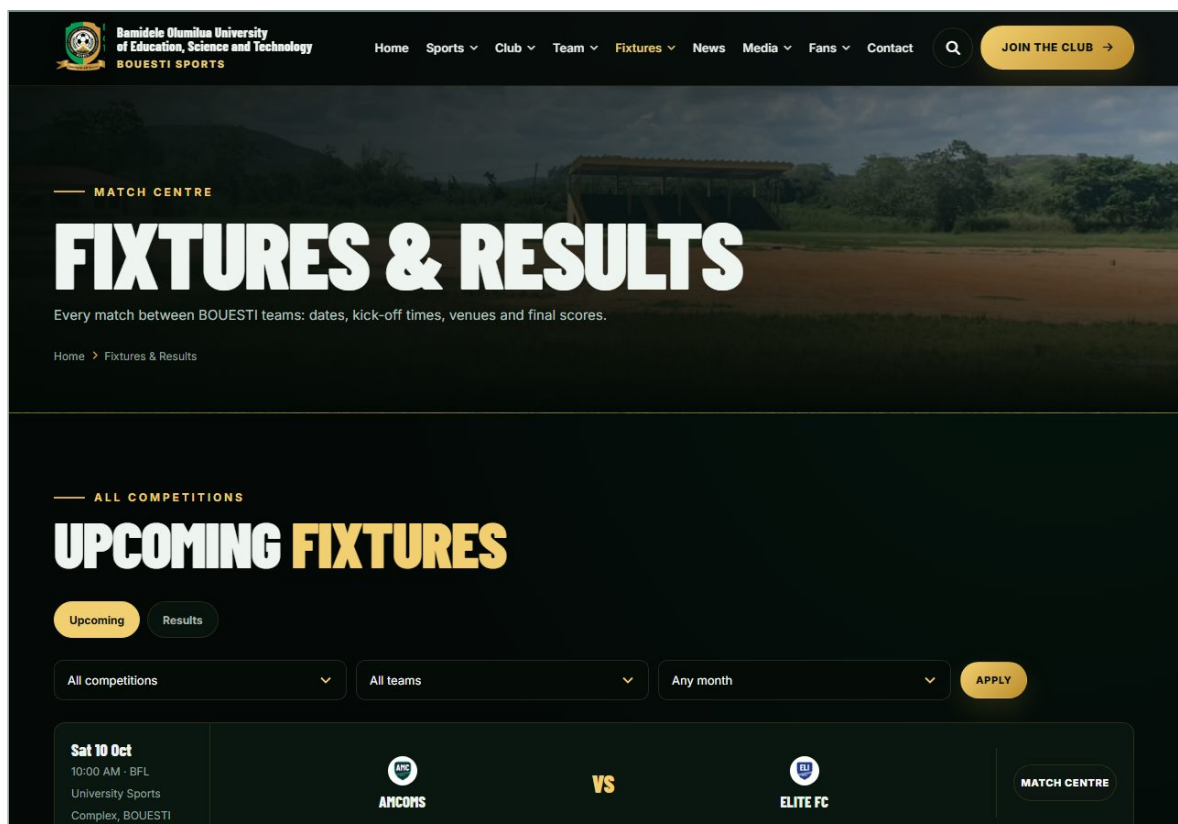


Figure 4.3: Fixtures and results with competition, team and month filters



Bamidele Olumilua University
of Education, Science and Technology
BOUESTI SPORTS

[Home](#)
[Sports](#)
[Club](#)
[Team](#)
[Fixtures](#)
[News](#)
[Media](#)
[Fans](#)
[Contact](#)



JOIN THE CLUB →

STANDINGS

LEAGUE TABLE

Standings are calculated automatically from completed matches: 3 points for a win, 1 for a draw.

Home > League Table

2026/2027

BOUESTI FOOTBALL LEAGUE TABLE

#	TEAM	P	W	D	L	GF	GA	GD	FORM	PTS
1	 CSC Elites	5	3	2	0	7	3	+4	W W D D W	11
2	 Amapro FC	5	3	1	1	10	4	+6	L W W D W	10
3	 AMCOMS	5	2	1	2	6	7	-1	L W D W L	7

Figure 4.4: Standings computed automatically from completed fixtures



Bamidele Olumilua University
of Education, Science and Technology
BOUESTI SPORTS

[Home](#)
[Sports](#)
[Club](#)
[Team](#)
[Fixtures](#)
[News](#)
[Media](#)
[Fans](#)
[Contact](#)



JOIN THE CLUB →

BOUESTI FOOTBALL LEAGUE • MATCHDAY 3

AMAPRO FC VS SOVEREIGNTY FC

Saturday, 05 September 2026 • 10:00 AM • University Sports Complex, BOUESTI

Home > Match Centre



AMAPRO FC

3-1

FULL TIME



SOVEREIGNTY FC

Referee: Mr. Olusegun Fadare • Attendance: 840

MINUTE BY MINUTE

Figure 4.5: Match centre with score and match events

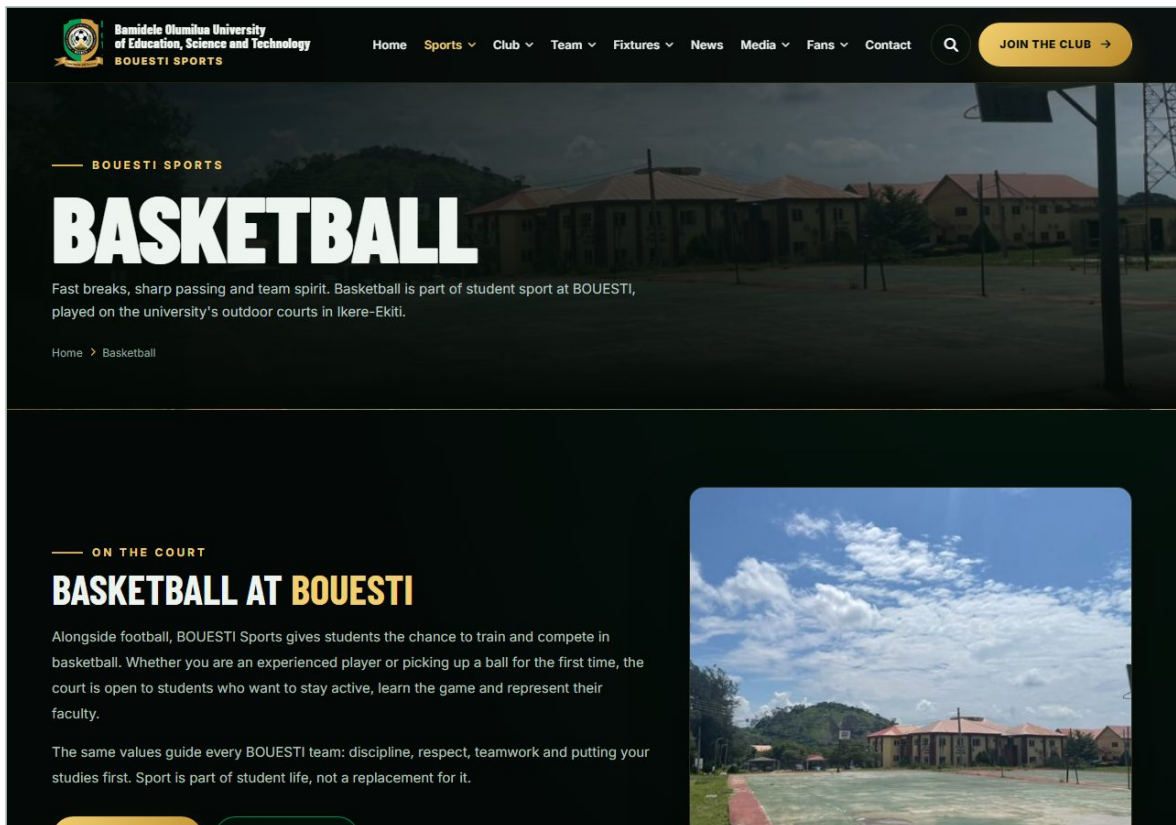


Figure 4.6: Basketball page

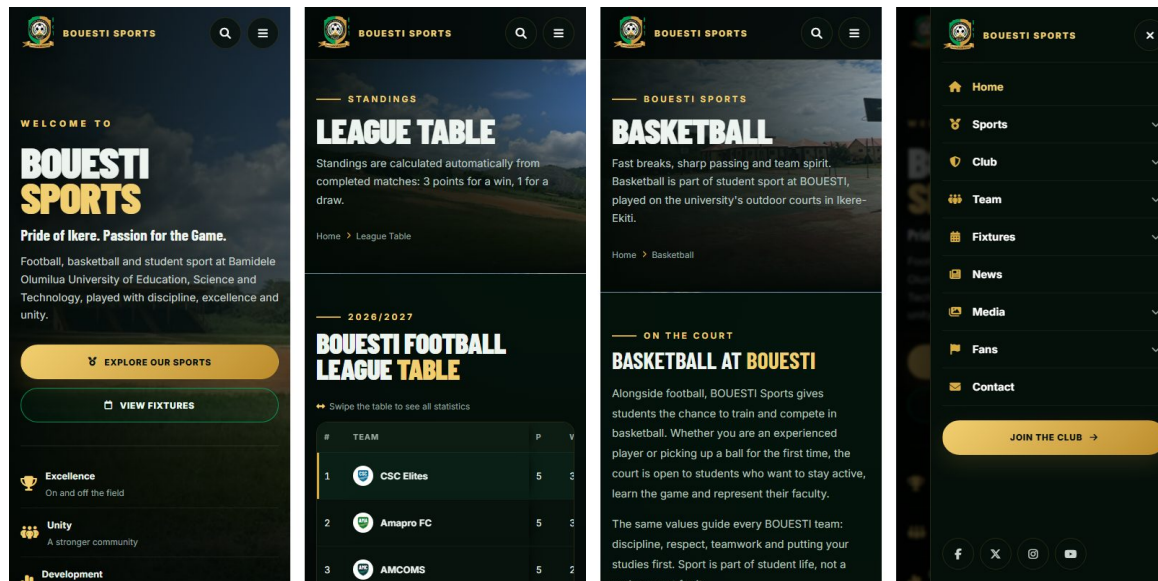


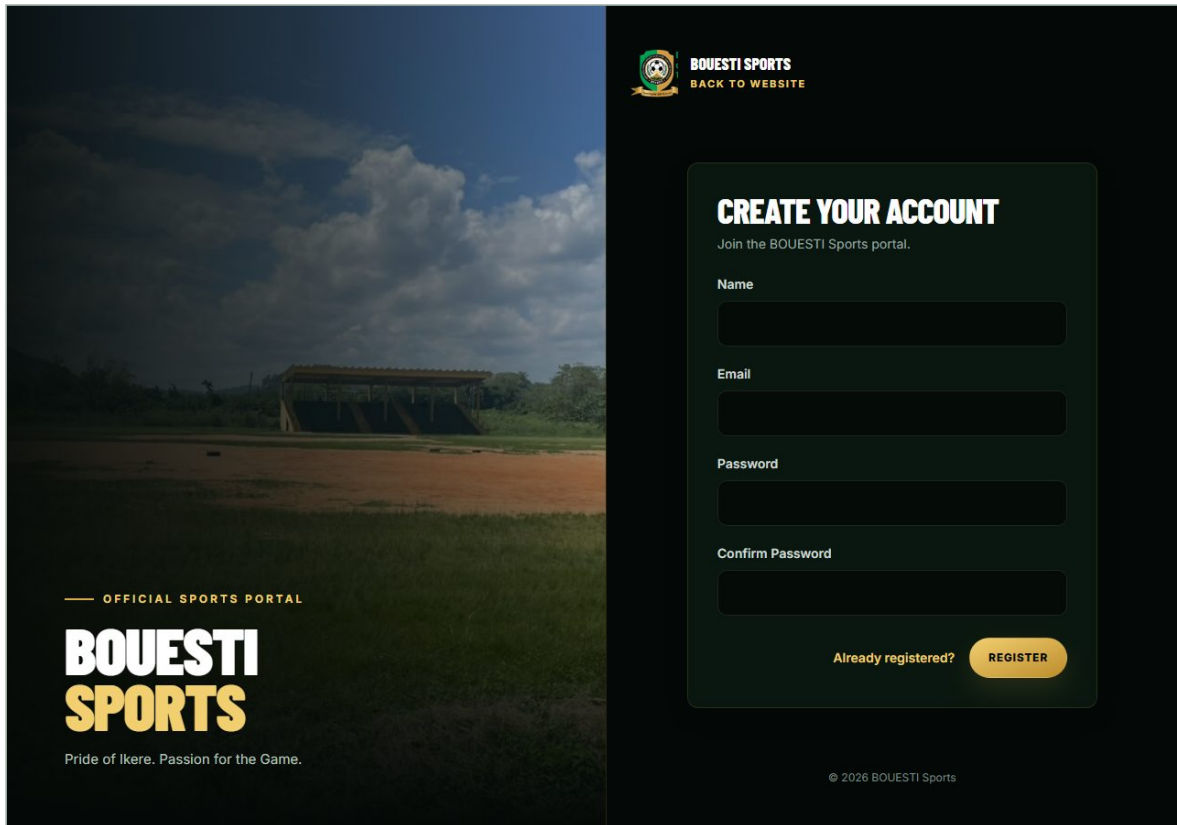
Figure 4.7: Mobile views — homepage, standings, basketball page and navigation menu

4.6 Authentication and Authorisation

Users register with a name, email and confirmed password (Figure 4.8) and log in through the login page (Figure 4.9). Passwords are hashed with bcrypt. Every administrative route passes through the *auth* and *admin* middleware; an administrator logging in is taken to the dashboard, an ordinary user to their account page, and an ordinary user who tries an administrative URL receives an HTTP 403 (forbidden) response — hiding a button is never relied upon. Table 4.3 summarises the roles.

Table 4.3: Roles and permissions

Role	Permissions
Administrator	Manage seasons, competitions, teams, players, fixtures, scores, match events, line-ups, news, media, staff, trial applications and messages.
Registered user (participant)	Log in, manage own profile and password; all public functions.
Visitor (spectator)	View schedules, results, standings, teams, players, news and media; apply for trials; send messages.



The registration page features a dark theme with a background image of a sports field. On the left, the text "OFFICIAL SPORTS PORTAL" is above the "BOUESTI SPORTS" logo, with the tagline "Pride of Ikere. Passion for the Game." below it. On the right, a "CREATE YOUR ACCOUNT" form includes fields for Name, Email, Password, and Confirm Password. A "REGISTER" button is at the bottom right of the form, with a link "Already registered?" to the left. The footer shows the copyright "© 2026 BOUESTI Sports".

BOUESTI SPORTS
BACK TO WEBSITE

CREATE YOUR ACCOUNT

Join the BOUESTI Sports portal.

Name

Email

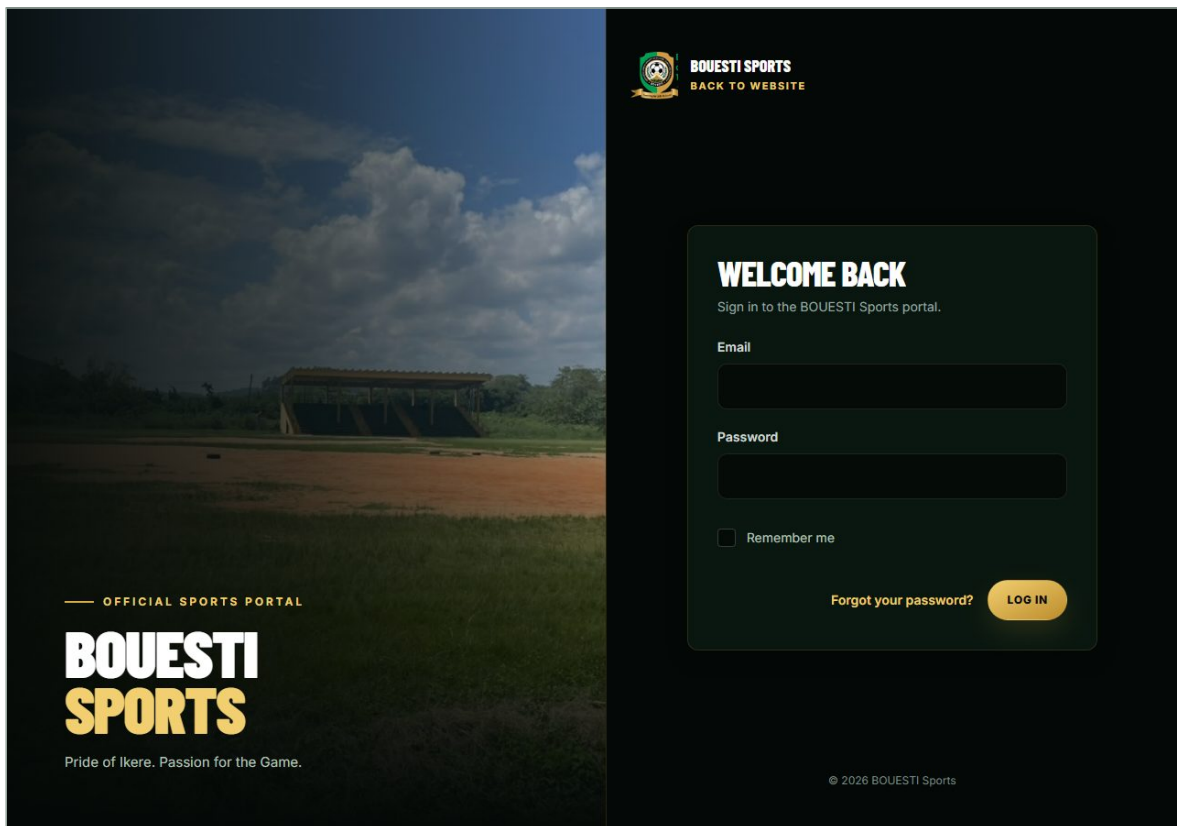
Password

Confirm Password

Already registered? [REGISTER](#)

© 2026 BOUESTI Sports

Figure 4.8: User registration page



The login page has the same layout as the registration page. On the right, a "WELCOME BACK" form includes fields for Email and Password. A "Remember me" checkbox is below the password field. A "LOG IN" button is at the bottom right of the form, with a link "Forgot your password?" to the left. The footer shows the copyright "© 2026 BOUESTI Sports".

BOUESTI SPORTS
BACK TO WEBSITE

WELCOME BACK

Sign in to the BOUESTI Sports portal.

Email

Password

☐ Remember me

Forgot your password? [LOG IN](#)

© 2026 BOUESTI Sports

Figure 4.9: Login page

4.7 Team and Athlete Management

Teams are listed publicly with their crests (Figure 4.10); each squad is grouped by position (Figure 4.11) and each player has a profile with statistics derived from match events (Figure 4.12). In the dashboard, administrators search and filter players (Figure 4.13) and manage each team's details, squad and record (Figure 4.14). Shirt numbers must be unique within a team. Teams and players that have played are archived (soft-deleted) rather than destroyed, so historical results keep their names. Private data such as matriculation numbers appear only in the dashboard.



Figure 4.10: Teams directory

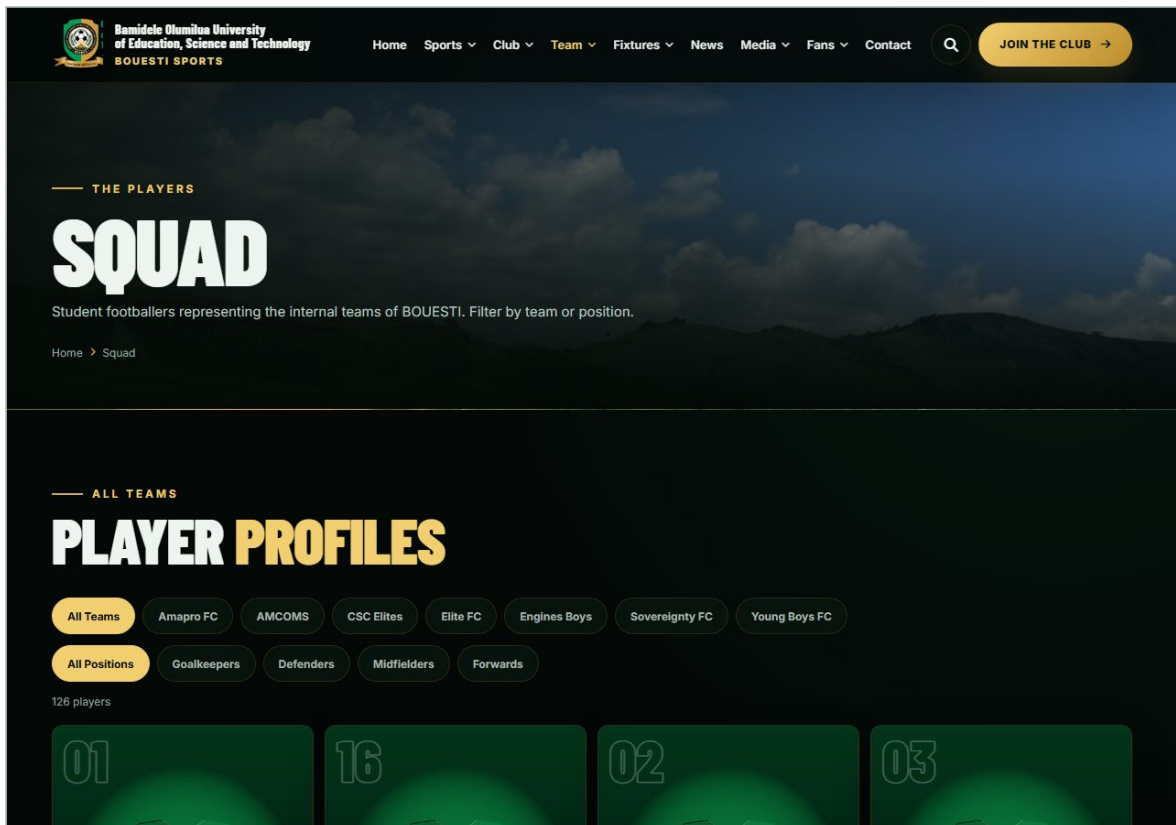


Figure 4.11: Squad page with team and position filters

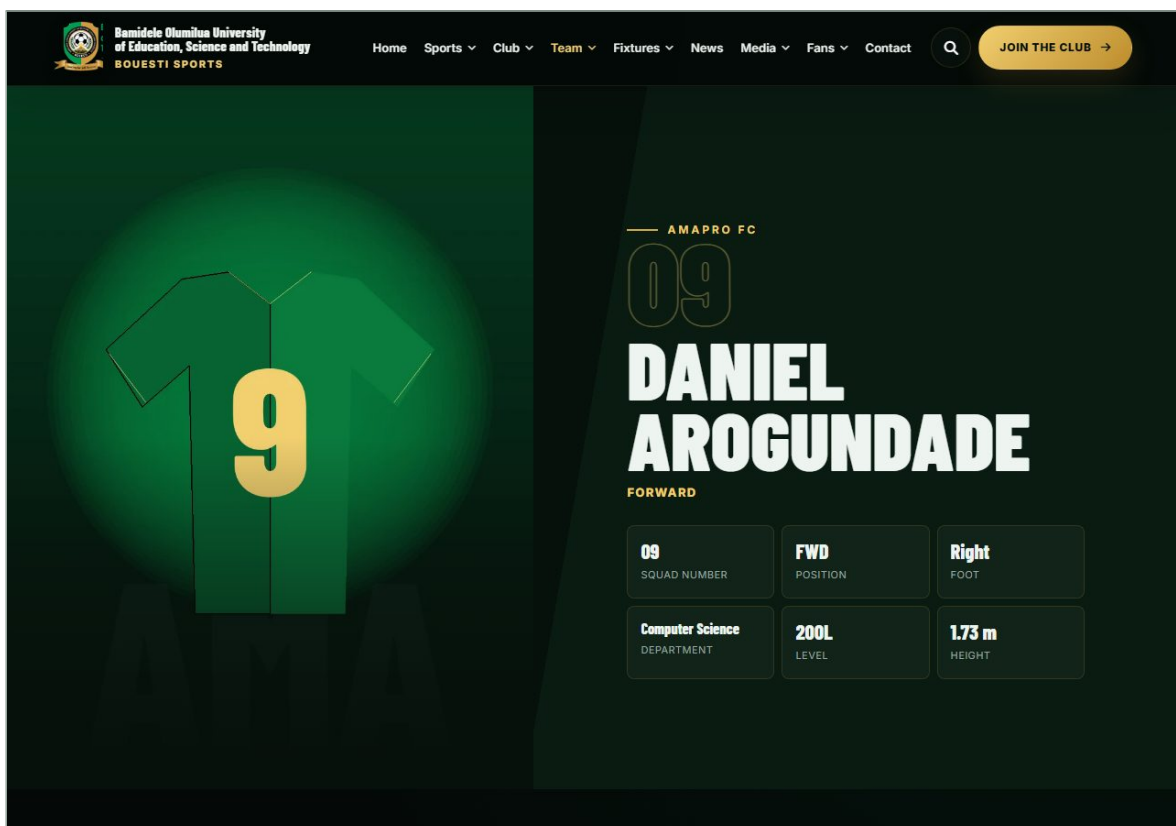


Figure 4.12: Player profile with statistics

BOUESTI FC
ADMIN PANEL

Dashboard

FOOTBALL

Seasons

Competitions

Teams

Players

MATCHES

Fixtures

Results

Match Events

CONTENT

News

Gallery

Videos

PEOPLE

Management & Staff

View website

PLAYERS

VIEW SITE

Portal Administrator

PLAYERS

Student players across all BOUESTI teams. Matric numbers are visible to admins only.

ADD PLAYER

All teams

All positions

FILTER

PLAYER	#	TEAM	POSITION	MATRIC NO.	STATUS			
1 Gbenga Adebayo	1	Amapro FC	Goalkeeper	80U/24/1443	ACTIVE			
16 Tobiloba Adewale	16	Amapro FC	Goalkeeper	80U/24/6299	ACTIVE			
2 Precious Lawal	2	Amapro FC	Defender	80U/25/7543	ACTIVE			
3 Ebuka Fatunla	3	Amapro FC	Defender	80U/25/8189	ACTIVE			
4 Favour Adegoke	4	Amapro FC	Defender	80U/24/3978	ACTIVE			
5 Godwin Eze	5	Amapro FC	Defender	80U/24/4512	ACTIVE			
12 Kunle Adeleke	12	Amapro FC	Defender	80U/23/5724	ACTIVE			
15 Emmanuel Anyanwu	15	Amapro FC	Defender	80U/23/9837	ACTIVE			
6 Oreoluwa Oluwadare	6	Amapro FC	Midfielder	80U/23/8647	ACTIVE			

Figure 4.13: Player management in the administration dashboard

BOUESTI FC
ADMIN PANEL

Dashboard

FOOTBALL

Seasons

Competitions

Teams

Players

MATCHES

Fixtures

Results

Match Events

CONTENT

News

Gallery

Videos

PEOPLE

Management & Staff

View website

AMAPRO FC

VIEW SITE

Portal Administrator

< Back

AMAPRO FC

ADD PLAYER

EDIT TEAM

PUBLIC PAGE

5 PLAYED

3 WON

1 DRAWN

1 LOST

10 GOALS FOR

4 GOALS AGAINST

2 CLEAN SHEETS

SQUAD (18)

1 Gbenga Adebayo
Goalkeeper

16 Tobiloba Adewale
Goalkeeper

2 Precious Lawal
Defender

3 Ebuka Fatunla
Defender

4 Favour Adegoke
Defender

5 Godwin Eze
Defender

12 Kunle Adeleke
Defender

RECENT & UPCOMING MATCHES

All

Elite FC VS Amapro ...
21 Nov - 10:00 AM

Amapro ... VS Young B...
14 Nov - 10:00 AM

CSC Elites VS Amapro ...
07 Nov - 10:00 AM

Amapro ... VS Engines ...
31 Oct - 10:00 AM

Sovereig... VS Amapro ...
24 Oct - 10:00 AM

Amapro ... VS AMCOMS
17 Oct - 10:00 AM

Amapro ... VS Elite FC
03 Oct - 10:00 AM

Figure 4.14: Team management — details, squad and record

38

4.8 Event and Fixture Management

Administrators create seasons (only one can be current) and competitions within them, then schedule fixtures by choosing the competition, home and away teams, date, kick-off time, venue and matchday (Figure 4.15). The form rejects a team playing itself and, through the conflict rule described in Section 3.13.2, a team being scheduled for two active matches on the same day (Figure 4.16). Competitions and seasons that already have fixtures cannot be deleted. Changes to dates, times, venues or status are saved centrally and appear on every public view of the fixture.

BOUESTI FC
ADMIN PANEL

FIXTURES
Schedule matches between BOUESTI teams and record results.

NEW FIXTURE

All competitions All teams All statuses **FILTER**

DATE	HOME	SCORE	AWAY	COMPETITION	STATUS
Sat 22 Aug 2026 10:00 AM - MD 1	Elite FC	2-1	AMCOMS	BFL	FULL TIME
Sat 22 Aug 2026 1:00 PM - MD 1	Young Boys FC	0-0	Sovereignty FC	BFL	FULL TIME
Sat 22 Aug 2026 4:00 PM - MD 1	CSC Elites	2-0	Engines Boys	BFL	FULL TIME
Sat 29 Aug 2026 10:00 AM - MD 2	AMCOMS	2-1	Amapro FC	BFL	FULL TIME
Sat 29 Aug 2026 1:00 PM - MD 2	Engines Boys	0-0	Elite FC	BFL	FULL TIME
Sat 29 Aug 2026 4:00 PM - MD 2	CSC Elites	2-1	Young Boys FC	BFL	FULL TIME
Sat 05 Sep 2026 10:00 AM - MD 3	Amapro FC	3-1	Sovereignty FC	BFL	FULL TIME
Sat 05 Sep 2026 1:00 PM - MD 3	AMCOMS	1-1	Engines Boys	BFL	FULL TIME
Sat 05 Sep 2026 4:00 PM - MD 3	Elite FC	0-0	Young Boys FC	BFL	FULL TIME

Figure 4.15: Fixture management in the administration dashboard

BOUESTI FC
ADMIN PANEL

NEW FIXTURE VIEW SITE P Portal Administrator

BOUESTI Football League (2026/2027)

The season is taken from the competition.

Home team* Amapro FC Away team* Elite FC

Date* 10/10/2026 Kick-off time 01:00 PM

❗ Schedule conflict: AMCOMS v Elite FC is already on this date. A team can only play one match per day.

Venue e.g. BOUESTI Sports Complex Matchday

☐ Featured match
Highlighted on the homepage when it is the next match.

STATUS & RESULT
Scores are only saved for live or completed matches. Completed results update the league table automatically.

Status* Scheduled Referee Attendance

CANCEL CREATE FIXTURE

Figure 4.16: Scheduling conflict detected and rejected

4.9 Score and Result Management

Scores are entered on the fixture's administration page (Figure 4.17). Choosing status “Full time” requires both scores; choosing postponed or cancelled clears any score, so an unfinished match is never published as a result. Goal, assist, card and substitution events are recorded with the minute; once a team is chosen, only that team's players are listed, and the server rejects any event whose team is not in the fixture or whose player belongs to another team. As soon as the result is saved, the public result, the standings, team and player statistics, the homepage and the match centre all reflect it, because all are computed from the same fixture record.

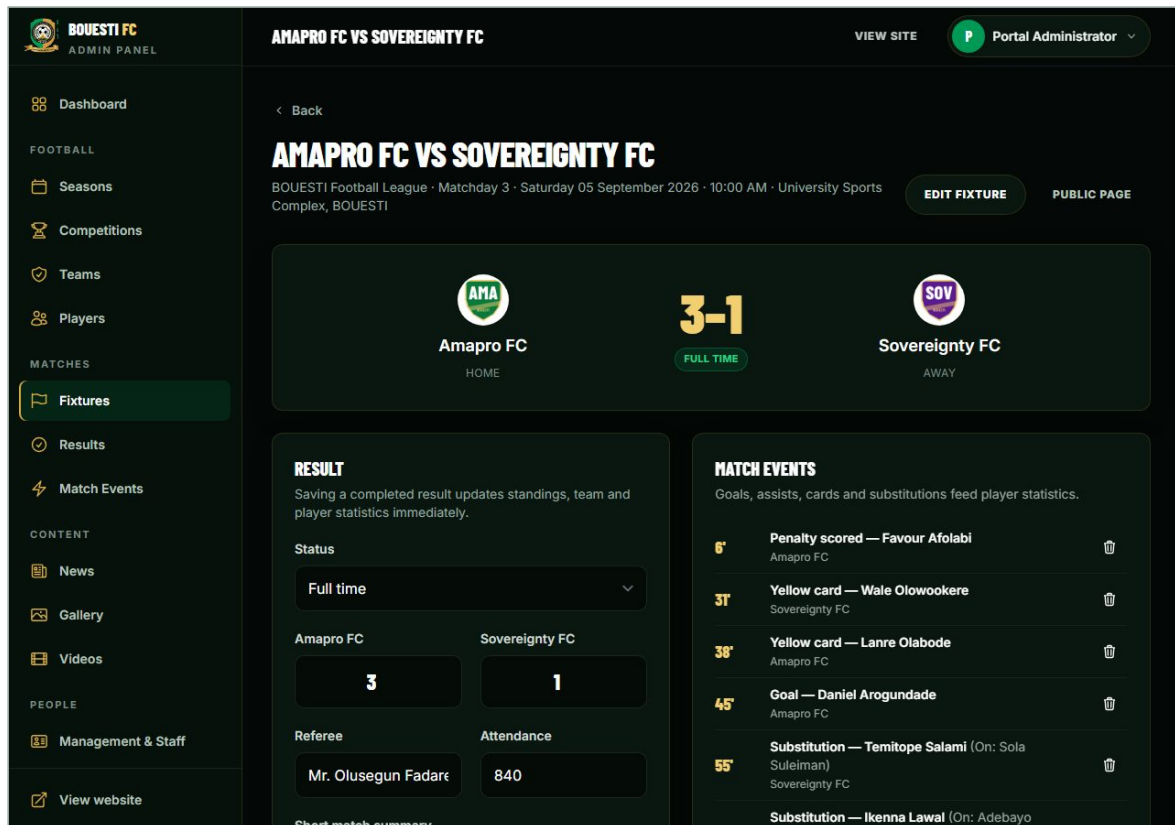


Figure 4.17: Result entry and match events

4.10 Administrative Dashboard

The dashboard (Figure 4.18) is the central management interface. It summarises total teams and players, upcoming and completed matches, active competitions and published news, alerts the administrator to new trial applications and unread messages, and lists upcoming fixtures and recent results with shortcuts to common tasks. The sidebar groups all functions under Football, Matches, Content and People.

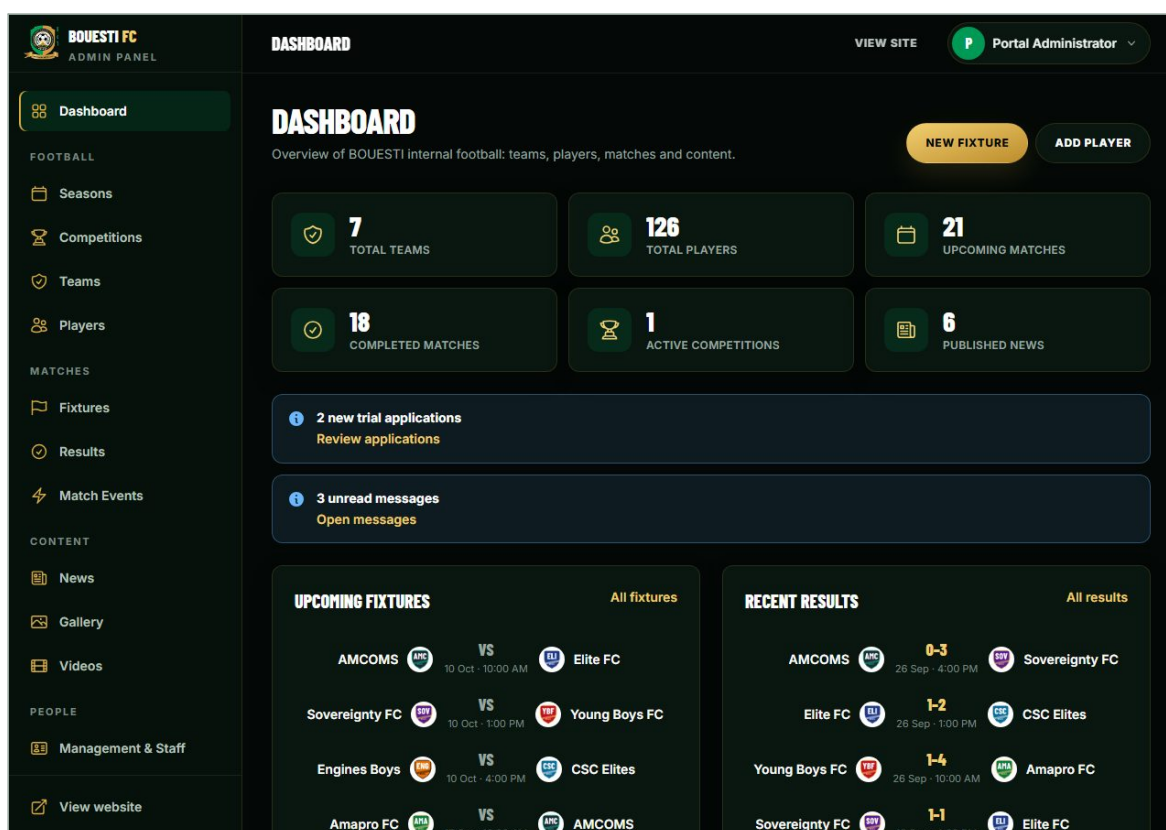


Figure 4.18: Administrative dashboard

4.11 Registration of Participants and Communication

Students apply for team trials through an online form (Figure 4.19) that captures their details, preferred position and experience; applications arrive in an administrative inbox (Figure 4.20) where they are reviewed and given a status. Contact messages follow the same pattern. Organisers publish news with a rich-text editor (Figure 4.21), including match reports linked to fixtures (Figure 4.22).

FOOTBALL TRIALS

A responsive registration experience ready to connect to a future Laravel backend.

Home > Football Trials

YOUR NEXT STEP

Want to play for a BOUESTI team? Register your interest below. Applications are reviewed by the football coordinators, who will contact you with trial dates.

ELIGIBILITY

Open to registered BOUESTI students. Bring your student ID to trials. Your matric number and contact details are only seen by the football administrators.

Full Name *

Email Address *

Phone Number *

Matric Number *

Department

Level *

Preferred Position *

Dominant Foot

Figure 4.19: Online trial application form

TRIAL APPLICATIONS

Students who applied through the Join the Team page.

ALL 4 PENDING 2 REVIEWING 1 ACCEPTED 1 REJECTED 0

Q Name, email or matric number FILTER

APPLICANT	POSITION	LEVEL	RECEIVED	STATUS
Aliyu Chukwu English Language	Defender	100 Level	25 Sep 2026	ACCEPTED
Toluwani Adeyemi Guidance and Counselling	Midfielder	300 Level	24 Sep 2026	REVIEWING
Samuel Fatunla Educational Management	Goalkeeper	100 Level	23 Sep 2026	PENDING
Sani Anyanwu English Language	Forward	200 Level	22 Sep 2026	PENDING

Figure 4.20: Trial applications inbox



BOUESTI FC

ADMIN PANEL

Dashboard

FOOTBALL

Seasons

Competitions

Teams

Players

MATCHES

Fixtures

Results

Match Events

CONTENT

News

Gallery

Videos

PEOPLE

Management & Staff

View website

NEW ARTICLE

[VIEW SITE](#) P Portal Administrator

[< Back](#)

NEW ARTICLE

CONTENT

Title *

Summary

One or two sentences shown on news cards. Leave empty to use the start of the article.

Article *

B

I

↶

↷

¶

”

<

>

:

≡

≡

≡

≡

PUBLISHING

☐ **Published**
Unticked articles stay as drafts.

Publish date

mm/dd/yyyy --:-- --

Leave empty to publish now. A future date schedules it.

☐ **Featured**

DETAILS

Category

No category

Related match

None

Links a match report to its fixture page.

Featured image

Figure 4.21: News editor



**Bamidele Olumilua University
of Education, Science and Technology**

BOUESTI SPORTS

[Home](#) [Sports](#) [Club](#) [Team](#) [Fixtures](#) [News](#) [Media](#) [Fans](#) [Contact](#)

[JOIN THE CLUB](#)

MATCH REPORT

AMAPRO FC CRUISE PAST YOUNG BOYS FC IN 4-1 WIN

[Home](#) > [Amapro FC cruise past Young Boys FC in 4...](#)

MATCH REPORT • 27 SEPTEMBER 2026

Amapro FC won 4-1 away at Young Boys FC on matchday 6 of the BOUESTI Football League. Goals: Daniel Arogundade 4', 16'; Ebuka Fatunla 14'; Favour Ajayi 40'; Stephen Adeleke 53'.

GOALS

- 4' — Daniel Arogundade (Amapro FC) scored
- 14' — Ebuka Fatunla (Amapro FC) scored after good work from Joshua Oyewole
- 16' — Daniel Arogundade (Amapro FC) scored after good work from Ademola Oladipo
- 40' — Favour Ajayi (Amapro FC) scored after good work from Daniel Arogundade
- 53' — Stephen Adeleke (Young Boys FC) scored after good work from Segun Olowookere

ARTICLE DETAILS

Category: Match Report

Published: 27 Sep 2026

By Portal Administrator

1 min read

Match centre: Young Boys FC vs Amapro FC

[← Back to News](#)

Figure 4.22: Published match report

44

4.12 System Integration

Integration occurs because the modules operate through shared data. The complete workflow below was executed end to end as an automated integration test (TC06, TC08 and TC09), confirming that data entered in one module is available to the next:

1. Create the season and competition (event).
2. Register participating teams and players.
3. Create fixtures using valid teams; conflicts are rejected.
4. The schedule is displayed publicly.
5. Conduct the match.
6. Record the score and match events against the correct fixture.
7. Mark the match full time to confirm the result.
8. The result, standings and player statistics are displayed to all viewers.

4.13 Test Cases and Results

Table 4.4 lists the functional test cases defined for the system with their observed results. Every test case was implemented as an automated test and executed with the Pest framework; the date of the final run was 8 October 2026.

Table 4.4: Functional test cases and results

ID	Test action	Expected outcome	Observed outcome	Status
TC01	Submit valid user registration	Account is created and the user is logged in.	Account stored; user authenticated and redirected to the dashboard.	Pass
TC02	Submit registration with empty name, invalid email and mismatched passwords	Submission rejected; fields identified.	Errors shown for name, email and password; no account created.	Pass
TC03	Log in with valid credentials	User reaches the appropriate interface.	User authenticated and redirected; administrator taken to the admin dashboard.	Pass
TC04	Log in with an invalid password	Access denied with an error.	User remains a guest; error shown.	Pass
TC05	Administrator creates a team	Team appears in team records.	Team stored with slug; offered in the fixture form; listed on the public Teams page.	Pass
TC06	Create a fixture with valid teams	Fixture stored and shown in schedule.	Fixture stored with its season; listed on the public Fixtures page.	Pass
TC07	Schedule a team for a second active fixture on the same day; schedule a team against itself	Conflict flagged; nothing saved.	Both rejected with explanatory messages; allowed on another day or after postponement; editing a fixture does not clash with itself.	Pass
TC08	Record a 2–1 score and a goal with assist for an existing match	Correct match score and statistics updated.	Score stored on the right fixture; scorer has 1 goal, assister 1 assist; incomplete scores and wrong-team players rejected.	Pass

ID	Test action	Expected outcome	Observed outcome	Status
TC09	Publish a completed result	Result visible on the results views.	Result shown on homepage and match centre; standings show the winner first with 3 points; unpublished news stays hidden (HTTP 404).	Pass
TC10	Unauthorised user opens an admin function	Protected operation denied.	Guest redirected to login; ordinary user receives HTTP 403; nothing created.	Pass
TC11	View schedule and all public pages in the browser	Pages displayed in readable form.	All 26 public URLs return HTTP 200 with an empty database and with the full sample season.	Pass
TC12	Edit an existing fixture	Updated information reflected in subsequent views.	New venue and date saved and shown on the public Fixtures page; old venue no longer shown.	Pass

Additional automated tests verified the integrity and security rules of the system (Table 4.5).

Table 4.5: Additional integrity and security tests

ID	Test	Observed outcome	Status
TC13	Duplicate shirt number in the same team / another team	Rejected in the same team; allowed across teams	Pass
TC14	Mark a second season as current	Only the new season is current	Pass
TC15	Delete a team that has played; delete a competition with fixtures	Team archived and history intact; competition deletion refused	Pass
TC16	Line-up containing a player from another team	Rejected; valid line-up saved	Pass
TC17	Upload a PHP file as a team logo; upload a 4000×3000 photo	PHP file rejected; photo stored as 2000×1500 WebP	Pass
TC18	Rich text containing script, onerror and javascript: links	Formatting kept; dangerous content removed	Pass
TC19	Trial and contact forms: valid, invalid and spam (honeypot) submissions	Stored; errors shown; spam rejected	Pass
TC20	CSV import with an invalid row; dry run	Whole import rolled back with line-numbered errors; dry run saves nothing	Pass
TC21	Generate sample season	126 players, 42 fixtures; every score equals its goal events; standings consistent	Pass

In total, **96 automated tests with 921 assertions were executed and all passed**. They include the tests above and the account tests supplied with Laravel Breeze (password reset, email verification, profile update and deletion), which confirm that standard account functions work correctly. The completed test sheet is in Appendix B.

4.14 Requirements Traceability

Table 4.6: Requirements traceability matrix

Requirement	Implementation module	Evidence
FR01 User registration	Breeze registration; trial applications	TC01, TC02, TC19
FR02 Authentication	Breeze login / logout; admin middleware	TC03, TC04, TC10
FR03 Team management	Admin\TeamController	TC05, TC15, TC17
FR04 Player management	Admin\PlayerController	TC08, TC13; Figure 4.13
FR05 Event management	Season and competition controllers	TC14, TC15
FR06 Fixture management	Admin\FixtureController, FixtureRequest	TC06, TC07, TC12; Figure 4.16
FR07 Score recording	Result, match-event and line-up modules	TC08, TC16
FR08 Result publication	Fixture status; LeagueTableService	TC09, TC21
FR09 Information access	Public controllers and views	TC11; Figures 4.1–4.7
FR10 Reporting	Standings, statistics, match centre, dashboard	TC09, TC21; Figure 4.18
FR11 Communication	Trial and contact modules	TC19; Figure 4.20
FR12 Content	News, gallery and video modules	TC17, TC18; Figure 4.21
FR13 Data import	FootballImporter	TC20

4.15 Usability Evaluation

Usability was evaluated against the design principles in Section 2.13 and by automated checks. An automated audit loaded every one of the 27 public pages in headless Microsoft Edge with true mobile emulation at ten widths — 320, 360, 375, 390, 414, 430, 768, 1024, 1280 and 1440 pixels (270 page loads). No page produced horizontal scrolling or overflow at any width; the only text reported below 11 pixels belonged to buttons that deliberately collapse to icons on phones while keeping accessible labels. Interactive elements — mobile menu, search, filters, form validation, the admin sidebar and confirmation dialogs — were exercised in the browser.

The interface also implements the accessibility practices identified in the literature: labelled form fields with error messages beside the field, keyboard-operable menus with visible focus indicators, text alternatives for images, semantic page landmarks and touch targets of at least 44 pixels on phones.

The user-satisfaction questionnaire in Appendix C had not been administered at the time of writing. It should be completed by representative organisers, participants and spectators after deployment, and the responses and sample size reported, before conclusions about user satisfaction are drawn.

4.16 Performance Evaluation

Response times of common operations were measured on the environment in Table 4.1 with the sample season loaded, using PHP's single-threaded development server. Each page was requested once to warm up and then ten times; Table 4.7 reports the median and the slowest of the ten requests. The login submission was measured once; it includes deliberate bcrypt password hashing and the redirect to the dashboard.

Table 4.7: Measured server response times (local environment)

Operation	URL	Median (ms)	Slowest (ms)
Homepage	/	76	134
Fixtures and results	/fixtures	67	99
Standings	/league-table	64	106
Match centre	/matches/7	72	85
Player profile	/players/daniel-arogundade	62	78
Basketball page	/basketball	43	51
Login page	/login	47	68
Login submission and redirect	POST /login	558 (single)	—
Administrative dashboard	/admin/dashboard	72	84
Result entry page	/admin/fixtures/7	82	96
Player management list	/admin/players	74	97

All common pages were generated in well under the one-second target of the performance requirement. These figures describe the local test environment; times on a production host will additionally depend on the server and network.

4.17 Discussion of Findings

The findings answer the research questions posed in Section 1.5:

1. **Registration and team management (RQ1).** Online registration, trial applications and linked team and player records replace scattered lists; validation rejected incomplete and duplicate data (TC01, TC02, TC05, TC13).
2. **Scheduling conflicts and delays (RQ2).** Central fixture records with conflict validation prevented a team from being double-booked or drawn against itself (TC07), and edits appeared immediately on every public view (TC12).
3. **Accuracy and accessibility of results (RQ3).** Scores are tied to their fixture and validated, and standings and statistics are computed from them, so the table always agrees with the results; the sample-season test confirmed that every score equals its recorded goals and the standings are consistent (TC08, TC09, TC21).

4. **Usability and effectiveness (RQ4).** All pages work from 320 to 1440 pixels without overflow, and all functional tests passed. Measured user satisfaction awaits the questionnaire.
5. **Administrative burden (RQ5).** Each fact is entered once and every dependent view updates automatically, eliminating manual table computation and repeated announcements. The magnitude of time saved should be measured after deployment.

The system has boundaries: it does not include live video streaming, online payment or external federation integration, and basketball is presented as information rather than managed competitions in this version.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This project addressed the problem of managing sports events through manual or fragmented administrative processes, in which registration, team management, fixture scheduling, score tracking and result publication become difficult to coordinate as participant and event volumes increase.

Chapter One established the background, problem, motivation, objectives, significance, scope and limitations. Chapter Two reviewed sports event management as an information-management problem, web-based systems, scheduling, score management, relational databases, three-tier and MVC architecture, frameworks, security and usability. Chapter Three presented the methodology, the analysis of the existing process, the requirements and the design of the system. Chapter Four described the implementation of the system as BOUESTI Sports and reported its evaluation: 96 automated tests with 921 assertions all passed, responsiveness was confirmed on all public pages at ten screen widths, and common pages were served in a median of 43–82 milliseconds locally.

5.2 Summary of Findings

- A centralised relational database effectively links participant, team, competition, fixture, score, match-event and result information.
- Computing standings and statistics from recorded results removes manual calculation errors and keeps every view consistent.
- Server-side validation prevents common data errors, including scheduling conflicts and events recorded against the wrong team.
- A responsive web interface makes schedules, results and standings accessible to participants and spectators on any device.
- Role-based access control and server-side authorisation keep administrative functions and private data away from ordinary users.
- Automated testing provides repeatable evidence that each requirement is implemented, and requirements traceability links every objective to its tests.
- The system is most appropriate as a focused solution for educational institutions and small organisations rather than a full professional federation platform.

5.3 Conclusion

The project concludes that a web-based sports event management system is an effective information-system response to the administrative challenges of manual sports administration. All three objectives were achieved: a user-friendly system for registration, scheduling and result tracking was designed (Objective 1); a functional platform enabling organisers to manage competitions, teams and scores, and users to access event information, was implemented and deployed as BOUESTI Sports (Objective 2); and the system's functionality, responsiveness and performance were evaluated, with user-satisfaction evaluation prepared for deployment (Objective 3).

The work also demonstrates that sports event management software should be designed around the workflow of its users. The most important functions are not isolated pages but connected processes: team information supports fixtures, fixtures support scores, and scores support results and standings. Human decisions — confirming eligibility, resolving disputes and validating official results — remain the responsibility of officials; the system makes those decisions easier to record and communicate.

5.4 Recommendations

1. Institutions adopting the system should train sports organisers and administrators using the user guide (Appendix F) before deployment.
2. User roles should be clearly defined, and only designated officials should hold administrator accounts.
3. The system should be hosted on reliable infrastructure with HTTPS and regular automated database backups before being used for important competitions.
4. A clear procedure should be established for verifying scores before matches are marked full time.
5. Verified squad data should be imported before launch, and participants' consent obtained before publishing photographs.
6. The user questionnaire (Appendix C) should be administered after deployment and its results used to guide improvements.
7. Data-retention and privacy practices for participant information should be documented.

5.5 Contribution to Knowledge

The project contributes a working, tested model for applying web technologies to small-scale sports event administration. Its contribution is not the claim that digital sports management is new, but the integration of the essential administrative functions

— registration, scheduling with conflict detection, score and event recording, derived standings and result publication — into a focused, institution-owned system suitable for an educational context. It also demonstrates requirements traceability from the identified problem through objectives, functional requirements, modules and executed test cases.

5.6 Data Quality and Record Integrity

A system can process information quickly but still produce poor outcomes if the information entered into it is incorrect. The implemented system therefore validates data at several levels: required fields and data types on every form, unique constraints for identifiers such as shirt numbers and email addresses, foreign keys so that a player references a valid team and an event a valid fixture, and cross-checks such as the conflict rule. These controls reduce, but do not eliminate, the need for human verification. An audit log recording who changed a score or fixture, and when, is recommended as a future extension.

5.7 Maintenance and Sustainability

The modular MVC structure, the migrations that define the database, the automated test suite and the installation guide reduce the difficulty of future maintenance. Maintenance should include defect correction, dependency and security updates, database backups and adaptation to new institutional requirements. Sustainability also depends on ownership: the institution should designate responsible administrators, define who may publish results and maintain procedures for correcting inaccurate information.

5.8 Suggested Future Work

- Full multi-sport management: a sport attribute on competitions, teams and fixtures so that basketball and other sports have fixtures, results and standings, with sport-specific scoring rules.
- Automatic fixture generation for round-robin leagues and knockout brackets for cup competitions.
- Live score updates and push notifications to subscribed users.
- Additional roles, such as team managers who submit their own line-ups for approval, and an audit log of administrative changes.
- Online registration payments where institutionally appropriate.

- Analytics dashboards and season-to-season comparisons of athlete and team performance.
- A mobile application or Progressive Web App with offline access.
- Integration with the institution's student information and identity-management systems.
- Cloud deployment with automated backups and operational monitoring.

REFERENCES

- Beck, K. (2002). *Test-Driven Development: By Example*. Boston: Addison-Wesley.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- Connolly, T., and Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Harlow: Pearson.
- Elmasri, R., and Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Hoboken, NJ: Pearson.
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Krasner, G. E., and Pope, S. T. (1988). A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80. *Journal of Object-Oriented Programming*, 1(3), 26–49.
- Laravel. (2025). *Laravel 12.x Documentation*. Retrieved from <https://laravel.com/docs/12.x>
- Laudon, K. C., and Laudon, J. P. (2020). *Management Information Systems: Managing the Digital Firm* (16th ed.). Harlow: Pearson.
- Marcotte, E. (2010, May 25). Responsive web design. *A List Apart*. Retrieved from <https://alistapart.com/article/responsive-web-design/>
- Nielsen, J. (1993). *Usability Engineering*. Boston: Academic Press.
- Okunbonade, A. A. (2026). *Project Proposal: Design and Implementation of a Web-Based Sports Event Management System*. Department of Computing and Information Science.
- OWASP Foundation. (2021). *OWASP Top 10:2021*. Retrieved from <https://owasp.org/Top10/>
- Pest. (2025). *Pest PHP Testing Framework Documentation*. Retrieved from <https://pestphp.com/docs>
- Pressman, R. S., and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). New York: McGraw-Hill Education.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Boston: Pearson.
- World Wide Web Consortium (W3C). (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Retrieved from <https://www.w3.org/TR/WCAG21/>

APPENDIX A: DATA DICTIONARY

Every table also has *created_at* and *updated_at* timestamps; soft-deletable tables have *deleted_at*.

Table A.1: users

Field	Type	Description / constraint
id	BIGINT, PK	Surrogate key
name	VARCHAR(255)	User's name
email	VARCHAR, unique	Login identifier
password	VARCHAR	bcrypt hash — never plain text
role	VARCHAR(20)	admin user (not mass-assignable)

Table A.2: seasons and competitions (events)

Field	Type	Description / constraint
seasons.name, slug	VARCHAR	e.g. 2026/2027
seasons.is_current	BOOLEAN	Only one season may be current
competitions.season_id	FK to seasons (restrict)	Owning season
competitions.name, short_name	VARCHAR	e.g. BOUESTI Football League / BFL
competitions.type	VARCHAR(20)	league cup friendly tournament
start_date, end_date, is_active	DATE / BOOLEAN	

Table A.3: teams

Field	Type	Description / constraint
id	BIGINT, PK	
name, slug	VARCHAR; unique	e.g. Amapro FC / amapro-fc
short_name	VARCHAR(12), null	Badge text
logo	VARCHAR, null	Path of optimised image
primary_color, secondary_color	CHAR(7), null	Hex colours
description, founded_year, captain_name, coach_name	TEXT / SMALLINT / VARCHAR, null	
is_active	BOOLEAN	
deleted_at	TIMESTAMP, null	Soft delete

Table A.4: players

Field	Type	Description / constraint
id	BIGINT, PK	
team_id	FK to teams (restrict)	Player's team
first_name, last_name, slug	VARCHAR; slug unique	
jersey_number	TINYINT, null	1–99; unique within team
position	VARCHAR(20)	goalkeeper defender midfielder forward
department, level	VARCHAR, null	Public
matric_number, state_of_origin	VARCHAR, null	Private — admin only
photo, dominant_foot, height, bio	VARCHAR / TEXT, null	
is_captain, is_featured, is_active	BOOLEAN	

Table A.5: fixtures (matches and results)

Field	Type	Description / constraint
id	BIGINT, PK	
competition_id, season_id	FK (restrict)	Season follows the competition
home_team_id, away_team_id	FK to teams (restrict)	Must differ; no same-day clash
match_date, kickoff_time	DATE, TIME null	
venue, referee	VARCHAR, null	
status	VARCHAR(20)	scheduled live completed postponed cancelled
home_score, away_score	TINYINT, null	Required for live/completed; cleared otherwise
matchday, attendance, featured, report	INT / BOOLEAN / TEXT	

Table A.6: match_events and fixture_players

Field	Type	Description / constraint
match_events.fixture_id	FK (cascade)	
match_events.team_id	FK to teams	Must be a team in the fixture
match_events.player_id, related_player_id	FK to players, null	Scorer / booked / player off; assister / player on
match_events.type	VARCHAR(20)	goal penalty_scored own_goal assist yellow_card red_card substitution penalty_missed
match_events.minute, additional_minute	TINYINT	e.g. 45 + 2
fixture_players.fixture_id, player_id, team_id	FK; unique (fixture, player)	Line-up entry

Field	Type	Description / constraint
fixture_players.is_starting, is_captain	BOOLEAN	

Table A.7: Content and communication tables (summary)

Table	Key fields
news_posts	title, slug, excerpt, content (sanitised HTML), featured_image, category, author_id, fixture_id, published_at, is_published
gallery_items / videos	title, image or video_url, category, fixture_id, team_id, is_featured
staff	name, slug, photo, role, type (management coaching), team_id, bio
trial_applications	full_name, email, phone, matric_number, department, level, preferred_position, experience, status, admin_notes
contact_messages	name, email, phone, subject, message, status (new read replied archived)

APPENDIX B: COMPLETED SYSTEM TEST SHEET

Date of final run: 8 October 2026. Tester: automated test suite (Pest 3), executed by the author. Environment: Table 4.1.

Table B.1: System test sheet

Test ID	Expected result	Observed result	Status
TC01	Valid registration accepted	Account created; user logged in	Pass
TC02	Invalid registration rejected	Errors for name, email, password; no account	Pass
TC03	Valid login accepted	Authenticated; correct interface	Pass
TC04	Invalid login rejected	Remains guest; error shown	Pass
TC05	Team created	Team stored; available in fixture form; public	Pass
TC06	Fixture created	Stored; shown in schedule	Pass
TC07	Conflict identified	Same-day clash and self-match rejected	Pass
TC08	Score recorded correctly	Correct fixture and statistics updated	Pass
TC09	Result published	Shown on result views and standings	Pass
TC10	Unauthorised action blocked	Login redirect / HTTP 403	Pass
TC11	Schedule readable	All public pages HTTP 200	Pass
TC12	Fixture edit reflected	New details shown publicly	Pass

APPENDIX C: PROPOSED USER QUESTIONNAIRE

The following questionnaire is to be administered after users interact with the deployed system. Actual responses should be collected and analysed before numerical findings are reported. Each statement is answered on the scale: Strongly Agree / Agree / Neutral / Disagree / Strongly Disagree.

1. I found the system easy to navigate.
2. The registration process was clear.
3. The team-management functions were understandable.
4. The fixture information was easy to find.
5. The score and result information was easy to understand.
6. The system reduced the difficulty of obtaining sports-event information.
7. I would be comfortable using the system during an actual sports event.
8. The system appears reliable for the core tasks it supports.
9. The interface is suitable for my device.
10. Overall, I am satisfied with the system.

Open comments: _____

APPENDIX D: MODULE SPECIFICATION

Table D.1: Module specification

Module	Specification	Implemented by
Authentication	Accepts credentials, validates them against stored hashed passwords, creates a session, provides logout, and returns non-revealing error messages.	Laravel Breeze; EnsureUserIsAdmin
Team management	Create, view, update and archive teams; associate players; preserve history of teams that have played.	Admin\TeamController, TeamRequest
Player management	Record athletes and their team; validate mandatory fields and unique shirt numbers; keep private fields admin-only.	Admin\PlayerController, PlayerRequest
Event	Seasons and competitions with type, dates and status; foundation for fixtures.	Season and Competition controllers
Fixture	Create, view, edit and change status of fixtures; reject self-matches and same-day conflicts.	Admin\FixtureController, FixtureRequest
Score and result	Scores, match events and line-ups against an existing fixture; result public when full time; corrections recompute everything.	FixtureResultRequest, MatchEvent and Lineup controllers
Reporting	Schedules, results, standings, statistics, match centre and dashboard summaries.	LeagueTableService, statistics services, public controllers

APPENDIX E: ACCEPTANCE CRITERIA

Table E.1: Acceptance criteria and status

Criterion	Status (evidence)
Registration is accepted when all mandatory fields are valid, and the record can be retrieved.	Met (TC01)
An invalid registration is rejected with a message explaining the correction required.	Met (TC02)
An administrator can create a team and then select it when creating a fixture.	Met (TC05)
A fixture cannot be saved with an invalid team reference or a scheduling conflict.	Met (TC07)
A schedule update is reflected wherever the fixture is displayed.	Met (TC12)
A score entered for one match is not displayed as the score of another match.	Met (TC08)
An incomplete result is not shown as an official public result.	Met (TC08, TC09)
An ordinary viewer cannot perform administrative create, update or delete operations.	Met (TC10)
Core pages remain readable on desktop and mobile-sized browser windows.	Met (Section 4.15)
Database records remain available after the application is restarted.	Met (persistent MySQL database)

APPENDIX F: USER AND INSTALLATION GUIDE

F.1 Visitors and participants. Open the website in any browser. Use the menu (the menu button on phones) to reach Sports, Club, Team, Fixtures, News, Media and Fans pages, and the search button to find any page. On the Fixtures page, switch between Upcoming and Results and filter by competition, team or month; select *Match Centre* for a match's events and line-ups. To create an account, choose Register; to apply for a team, open *Join the Club*.

F.2 Administrators. Log in at */login*; you are taken to */admin/dashboard*.

1. Seasons > New season (tick *Current season*); Competitions > New competition (type League for standings).
2. Teams: add or edit each team; Players > Add player, choosing the team and position.
3. Fixtures > New fixture: choose competition, teams, date, kick-off time and venue. A team cannot be booked twice on the same day.
4. After the match, open the fixture, choose *Full time*, enter both scores and save — standings update immediately. Then add match events and tick the line-ups.
5. Use News, Gallery and Videos for content, and Trial Applications and Messages for the inboxes. Deleting asks for confirmation; teams and players are archived, not destroyed.

F.3 Installation. Requirements: PHP 8.2+ with GD and PDO MySQL extensions, Composer 2, Node.js 18+, MySQL 8 or MariaDB 10.4+.

```
composer install --no-dev --optimize-autoloader
npm install && npm run build
cp .env.example .env && php artisan key:generate
#   edit .env: APP_URL, DB_*, ADMIN_EMAIL, ADMIN_PASSWORD, MAIL_*, SEED_SAMPLE_DATA=false
php artisan migrate --seed          # season, league, teams, administrator
php artisan storage:link
php artisan football:import         # load verified CSV data (use --dry-run first)
php artisan test                    # run the automated tests
```

APPENDIX G: SELECTED SOURCE CODE

G.1 Administrator access middleware (app/Http/Middleware/EnsureUserIsAdmin.php)

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserIsAdmin
{
    /** Only users with the admin role may reach /admin routes. */
    public function handle(Request $request, Closure $next): Response
    {
        abort_unless($request->user()?->isAdmin(), 403, 'This area is for portal
        administrators only.');
```

G.2 Fixture validation, including the scheduling-conflict rule

(app/Http/Requests/Admin/FixtureRequest.php)

```
<?php

namespace App\Http\Requests\Admin;

use Illuminate\Foundation\Http\FormRequest;
use App\Models\Fixture;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Validator;

class FixtureRequest extends FormRequest
{
    /** Access is enforced by the `admin` route middleware. */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }

    public function rules(): array
    {
        $scored = Rule::requiredIf(in_array($this->input('status'),
            Fixture::SCORED_STATUSES, true));

        return [
            'competition_id' => ['required', 'integer', 'exists:competitions,id'],
            'home_team_id' => ['required', 'integer', Rule::exists('teams',
                'id')->whereNull('deleted_at')],
            'away_team_id' => ['required', 'integer', 'different:home_team_id',
                Rule::exists('teams', 'id')->whereNull('deleted_at')],
            'match_date' => ['required', 'date'],
            'kickoff_time' => ['nullable', 'date_format:H:i'],
            'venue' => ['nullable', 'string', 'max:120'],
            'matchday' => ['nullable', 'integer', 'min:1', 'max:200'],
            'status' => ['required', Rule::in(array_keys(Fixture::STATUSES))],
            'home_score' => [$scored, 'nullable', 'integer', 'min:0', 'max:99'],
            'away_score' => [$scored, 'nullable', 'integer', 'min:0', 'max:99'],
            'referee' => ['nullable', 'string', 'max:80'],
            'attendance' => ['nullable', 'integer', 'min:0', 'max:1000000'],
            'featured' => ['boolean'],
            'report' => ['nullable', 'string', 'max:5000'],
        ];
    }
}

/** A team cannot be scheduled for two active fixtures on the same day. */
```

```

public function after(): array
{
    return [function (Validator $validator) {
        if ($validator->errors()->isEmpty() || in_array($this->input('status'),
            Fixture::INACTIVE_STATUSES, true)) {
            return;
        }

        $teams = [(int) $this->input('home_team_id'), (int)
            $this->input('away_team_id')];
        $slash = Fixture::query()
            ->whereDate('match_date', $this->input('match_date'))
            ->whereNotIn('status', Fixture::INACTIVE_STATUSES)
            ->where(fn ($q) => $q->whereIn('home_team_id',
                $teams)->orWhereIn('away_team_id', $teams))
            ->when($this->route('fixture'), fn ($q, $fixture) =>
                $q->whereKeyNot($fixture->getKey()))
            ->with('homeTeam', 'awayTeam')
            ->first();

        if ($slash) {
            $validator->errors()->add('match_date', "Schedule conflict:
                {$slash->homeTeam?->name} v {$slash->awayTeam?->name} is already on
                this date. A team can only play one match per day.");
        }
    }]);
}

public function messages(): array
{
    return [
        'away_team_id.different' => 'A team cannot play against itself. Choose a
            different away team.',
        'home_score.required' => 'Enter the home score for a live or completed match.',
        'away_score.required' => 'Enter the away score for a live or completed match.',
    ];
}

protected function prepareForValidation(): void
{
    $this->merge([
        'featured' => $this->boolean('featured'),
        'kickoff_time' => $this->filled('kickoff_time') ? substr((string)
            $this->input('kickoff_time'), 0, 5) : null,
    ]);
}
}

```

G.3 Standings computation (app/Services/LeagueTableService.php)

```

<?php

namespace App\Services;

use App\Models\Competition;
use App\Models\Fixture;
use App\Models\Season;
use App\Models\Team;
use Illuminate\Support\Collection;

/**
 * Builds standings from COMPLETED fixtures – points are never stored.
 * Win 3, draw 1, loss 0. Sorted by points, goal difference, goals for, then name.
 */
class LeagueTableService
{
    public const WIN = 3;
    public const DRAW = 1;

    /**
     * @return Collection<int, object{position:int, team:Team, played:int, won:int,
        drawn:int, lost:int,

```

```

*           goals_for:int, goals_against:int,
    goal_difference:int, points:int, form:array}>
*/
public function generate(Competition $competition, ?Season $season = null): Collection
{
    $fixtures = Fixture::query()
        ->where('competition_id', $competition->id)
        ->when($season, fn ($q) => $q->where('season_id', $season->id))
        ->get(['id', 'home_team_id', 'away_team_id', 'status', 'home_score',
            'away_score', 'match_date', 'kickoff_time']);

    // Every team drawn into the competition appears, even before its first result.
    $teamIds = $fixtures->pluck('home_team_id')->merge($fixtures->pluck('away_team_id'))
        ->unique();
    if ($teamIds->isEmpty()) {
        return collect();
    }

    $rows = Team::withTrashed()->whereIn('id', $teamIds)->get()->mapWithKeys(fn (Team
        $team) => [
            $team->id => (object) [
                'position' => 0, 'team' => $team, 'played' => 0, 'won' => 0, 'drawn' => 0,
                'lost' => 0,
                'goals_for' => 0, 'goals_against' => 0, 'goal_difference' => 0, 'points' =>
                0, 'form' => [],
            ],
        ]);

    $completed = $fixtures
        ->filter(fn (Fixture $f) => $f->status === 'completed' && $f->home_score !==
            null && $f->away_score !== null)
        ->sortBy(fn (Fixture $f) => $f->match_date->format('Y-m-d') . ' ' .
            $f->kickoff_time . sprintf('%08d', $f->id));

    foreach ($completed as $fixture) {
        $this->record($rows[$fixture->home_team_id], $fixture->home_score,
            $fixture->away_score);
        $this->record($rows[$fixture->away_team_id], $fixture->away_score,
            $fixture->home_score);
    }

    return $rows->values()
        ->each(function ($row) {
            $row->goal_difference = $row->goals_for - $row->goals_against;
            $row->form = array_slice($row->form, -5);
        })
        ->sort(fn ($a, $b) => [$b->points, $b->goal_difference, $b->goals_for,
            $a->team->name]
            <=> [$a->points, $a->goal_difference, $a->goals_for, $b->team->name])
        ->values()
        ->each(fn ($row, $i) => $row->position = $i + 1);
}

/** A team's current position in a competition, or null if it is not in the table. */
public function positionOf(Team $team, Competition $competition, ?Season $season =
    null): ?object
{
    return $this->generate($competition, $season)->first(fn ($row) => $row->team->id
        === $team->id);
}

private function record(object $row, int $for, int $against): void
{
    $row->played++;
    $row->goals_for += $for;
    $row->goals_against += $against;

    if ($for > $against) {
        $row->won++;
        $row->points += self::WIN;
        $row->form[] = 'W';
    } elseif ($for === $against) {

```

```

        $row->drawn++;
        $row->points += self::DRAW;
        $row->form[] = 'D';
    } else {
        $row->lost++;
        $row->form[] = 'L';
    }
}
}
}

```

G.4 Validation of match events against the fixture's teams

(app/Http/Requests/Admin/MatchEventRequest.php)

```

<?php

namespace App\Http\Requests\Admin;

use Illuminate\Foundation\Http\FormRequest;
use App\Models\MatchEvent;
use Illuminate\Validation\Rule;

class MatchEventRequest extends FormRequest
{
    /** Access is enforced by the `admin` route middleware. */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }

    public function rules(): array
    {
        $fixture = $this->route('fixture');
        $teamId = $this->integer('team_id');
        $type = $this->input('type');

        // Players must belong to the chosen team, which must be playing in this fixture.
        $teamPlayer = Rule::exists('players', 'id')->where('team_id',
            $teamId)->whereNull('deleted_at');

        return [
            'type' => ['required', Rule::in(array_keys(MatchEvent::TYPES))],
            'team_id' => ['required', 'integer', Rule::in([$fixture->home_team_id,
                $fixture->away_team_id])],
            'player_id' => ['required', 'integer', $teamPlayer],
            'related_player_id' => [
                Rule::requiredIf($type === 'substitution'),
                Rule::prohibitedIf(! in_array($type, ['goal', 'penalty_scored',
                    'substitution'], true)),
                'nullable', 'integer', 'different:player_id', $teamPlayer,
            ],
            'minute' => ['required', 'integer', 'min:0', 'max:130'],
            'additional_minute' => ['nullable', 'integer', 'min:1', 'max:30'],
            'description' => ['nullable', 'string', 'max:255'],
        ];
    }

    public function messages(): array
    {
        return [
            'team_id.in' => 'Choose one of the two teams playing in this match.',
            'player_id.exists' => 'The selected player is not in the chosen team.',
            'related_player_id.exists' => 'The second player must be in the same team.',
            'related_player_id.required' => 'Choose the player coming on.',
            'related_player_id.different' => 'The two players must be different.',
            'related_player_id.prohibited' => 'Only goals and substitutions take a second
                player.',
        ];
    }

    public function attributes(): array
    {

```

```
        return ['player_id' => 'player', 'related_player_id' => 'second player', 'team_id'
                => 'team'];
    }
}
```